



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

严谨 · 勤奋 · 求实 · 创新

数字系统设计

10-稀疏加速器与NAS

贾弘洋 & 刘勇攀

2025/04/22



/ 授课计划 (Tentative)



- 1) Introduction of DNN
- 2) DNN的原理
- 3) Quantization (Lab 1 out)
- 4) Pruning (剪枝)
- 5) Kernel Computation & Computing Flow
- 6) Lab 1 Presentation
- 7) Accelerator & Case Study (TPU&Ascend) (Lab 2 out)
- 8) Mapping of DNN & Toolchain (Layer Fusion)
- 9) Data Movement & Storage Hierarchy (Compression)
- 10) 校庆+五一
- 11) Lab 2 Presentation
- 12) Codesign Sparse-aware Arch & NAS (Lab 3 out)
- 13) Compute-in Memory (CIM)
- 14) End2End Deployment & ALP
- 15) Training
- 16) Conclusion

Outline

- ❑ **Introduction to Neural Network Sparsity (Review)**
- ❑ **Spatial-Domain Sparsity Based NN Processor**
- ❑ **Temporal-Domain Sparsity Based NN Processor**
- ❑ **Neural Architecture Search**
- ❑ **Conclusion**

Where does sparsity come from

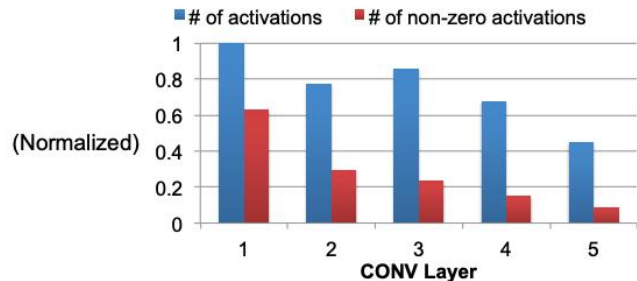
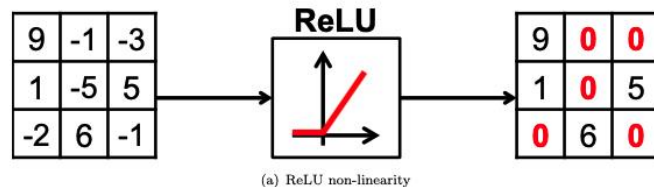
□ Activation Sparsity

□ ReLU function

□ Sample from another network

□ Weight Sparsity

□ Network Prune



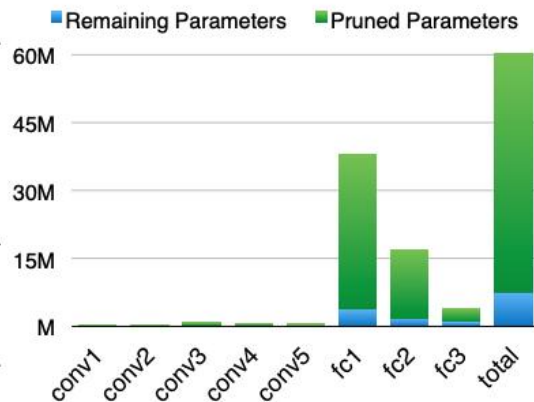
Benefits of Sparsity

❑ Reduce Storage Burden

❑ Reduce Computation Burden

Table 4: For AlexNet, pruning reduces the number of weights by $9\times$ and computation by $3\times$.

Layer	Weights	FLOP	Act%	Weights%	FLOP%
conv1	35K	211M	88%	84%	84%
conv2	307K	448M	52%	38%	33%
conv3	885K	299M	37%	35%	18%
conv4	663K	224M	40%	37%	14%
conv5	442K	150M	34%	37%	14%
fc1	38M	75M	36%	9%	3%
fc2	17M	34M	40%	9%	3%
fc3	4M	8M	100%	25%	10%
Total	61M	1.5B	54%	11%	30%



How to encode sparse data

❑ Bit Mask

Original Format

	Column			
	A	0	B	C
Row	0	0	0	0
	E	0	0	F

Compressed Format

Bitmask	1	0	1	1	0	0	0	0	1	0	0	1
Value	A	B	C	E	F							

❑ Run length encoding

Original Format

	Column			
	A	0	B	C
Row	0	0	0	0
	E	0	0	F

Compressed Format

	Run		Run		Run		...			
Value	0	A	1	B	0	C	4	E	2	F
	Level		Level		Level		...			

How to encode sparse data

❑ Bit Mask

Original Format

		Column			
Row	A	0	B	0	C
	0	0	0	0	0
	E	0	0	0	F

Compression Ratio?

Compressed Format

Bitmask	1	0	1	1	0	0	0	0	1	0	0	1
Value	A	B	C	E	F							

$$\frac{B_w}{pB_w + B_M}$$

❑ Run length encoding

Original Format

		Column			
Row	A	0	B	0	C
	0	0	0	0	0
	E	0	0	0	F

Compressed Format

	Run	Run	Run	...						
Value	0	A	1	B	0	C	4	E	2	F
	Level	Level	Level	...						

Which one is better? It depends

How to encode sparse data

❑ Compress Sparse Row (CSR)

Original Format

		Column		
Row	A	0	B	C
	0	0	0	0
	E	0	0	F

Compressed Format

Bounds (row)	0	3	3	5	
Index (col)	0	2	3	0	3
Value	A	B	C	E	F

❑ Compress Sparse Column (CSC)

Original Format

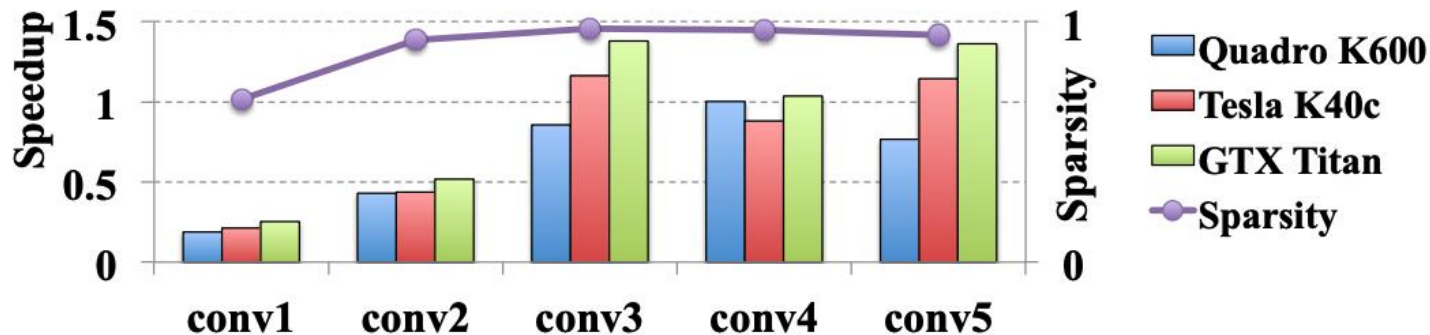
		Column		
Row	A	0	B	C
	0	0	0	0
	E	0	0	F

Compressed Format

Bounds (col)	0	2	2	3	5
Index (row)	0	2	0	0	2
Value	A	E	B	C	F

Why we need HW-SW codesign

- Sparsity does not directly translate to speedup and data compression



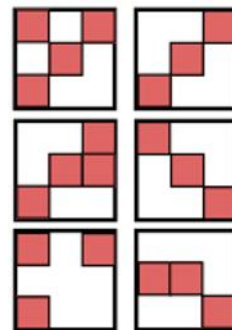
- **Domain-specific accelerator** is needed

Outline

- ❑ Introduction to Neural Network Sparsity
- ❑ **Spatial-Domain Sparsity Based NN Processor**
- ❑ Temporal-Domain Sparsity Based NN Processor
- ❑ Sparsity-Aware CIM NN Processor
- ❑ Conclusion

Two ways to utilize Sparsity

- Unstructured sparsity utilization
 - Sticker in VLSI 2018
 - Sticker-II in A-SSCC 2019
- Structured sparsity utilization
 - Sticker-T in ISSCC 2019



-1.39	0.06	1.56	0.36
0.36	-1.39	0.06	1.56
1.56	0.36	-1.39	0.06
0.06	1.56	0.36	-1.39

Challenge 1: Wide Sparsity Distribution

➤ Dimension 1: Different Network / Data Type

Weights

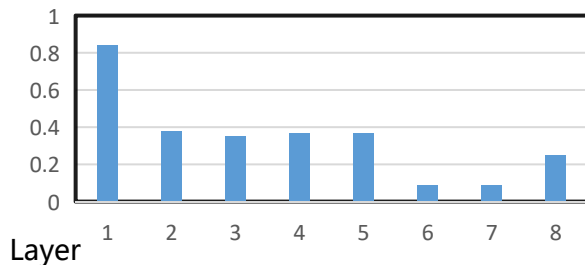
Network	Sparsity
LeNet-5	8%—66%
AlexNet	9%—84%
VGG	4%—58%

Activations

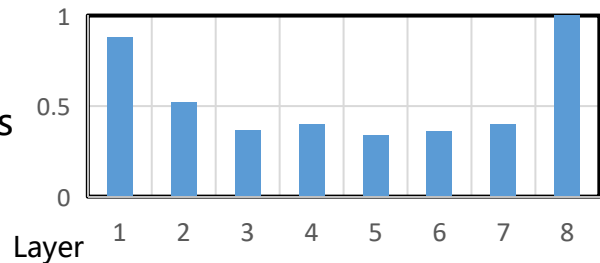
Network	Sparsity
LeNet-5	55%—82%
AlexNet	34%—88%
VGG	19%—89%

➤ Dimension 2: Different Layers in a Network

Alexnet
Weights
Sparsity

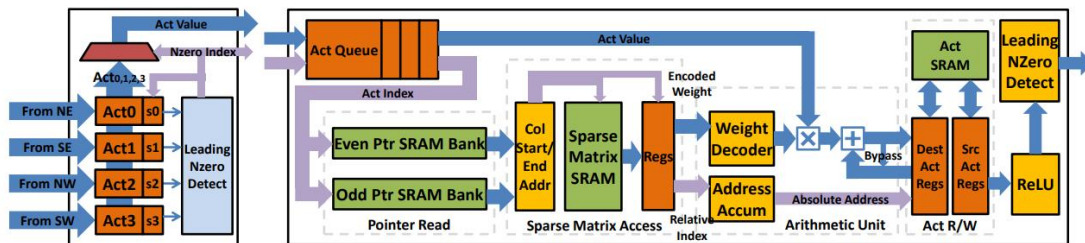
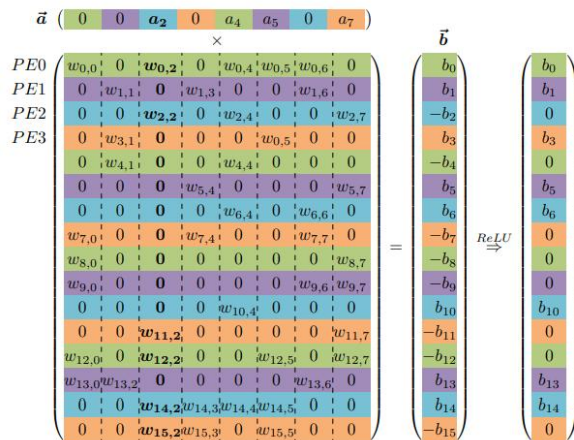


Alexnet
Activations
Sparsity



➤ Efficiently Process **All Sparse Situations** Is Important!

EIE: Sparse CNN Architecture



Han S, EIE (ISCA2016)

- Sparse Storage Format

- Compressed Sparse Column (CSC)

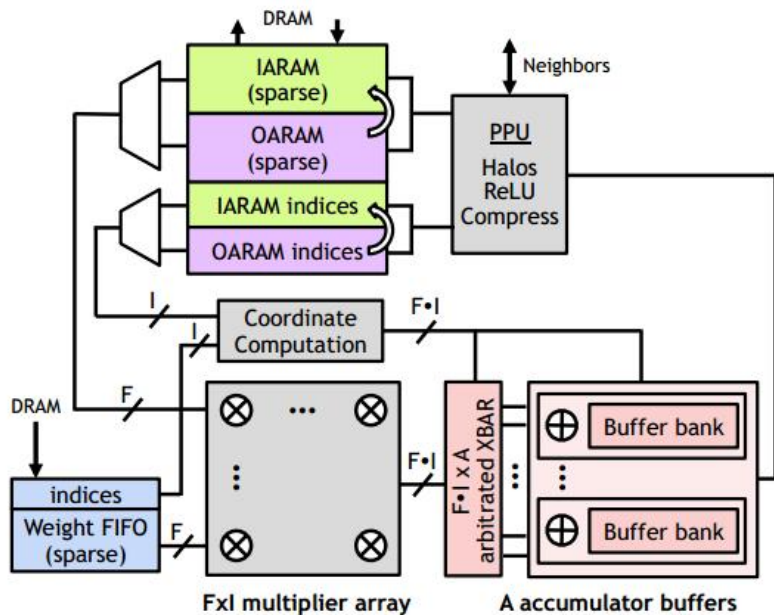
3400 × Energy Efficient than GeForce TITAN X

- Parallel PE Processing

19 × Energy Efficient than DaDianNao

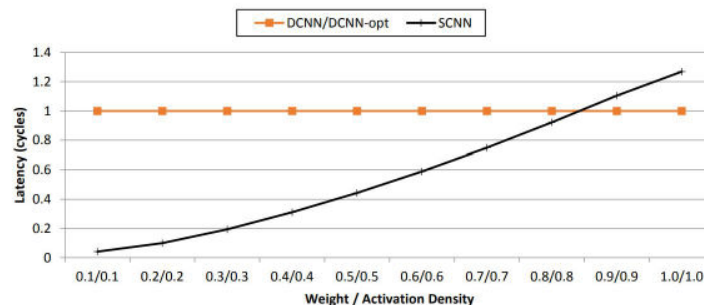
- Skip both clock cycles and energy

SCNN: Sparse CNN Architecture

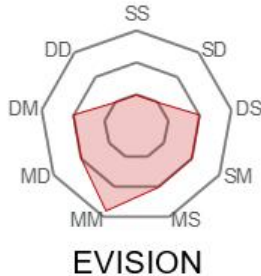
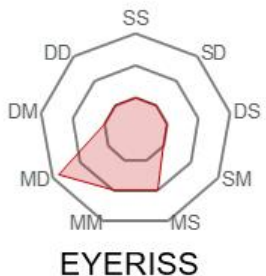
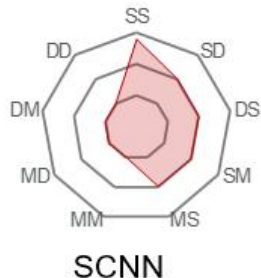


Parashar A, SCNN (ISCA2017)

- Index-Based Data Storage
 - Activation + Weight Sparse
- Sparse CNN Processing
 - Data Multiply + Index calculate
 - Hash-Based Accumulation Buffer



Previous Solution: Incomplete Sparsity Coverage



9 Typical sparsity mode

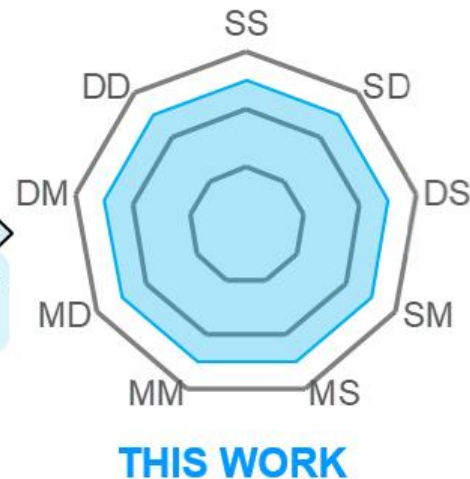
Weight	SD	MD	DD
	SM	MM	DM
	SS	MS	DS

Activation

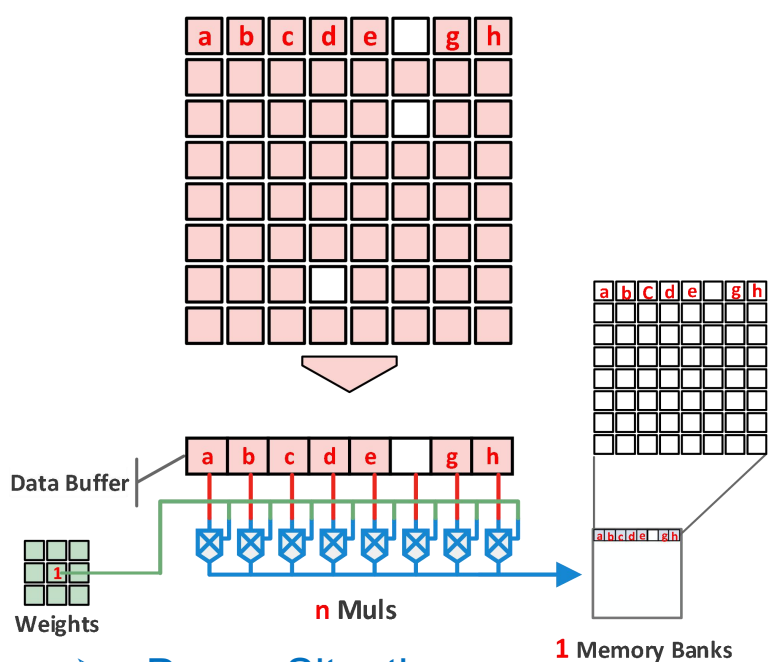
Design Target

9 sparsity mode
compatible

S: Sparse D: Dense M: Medium

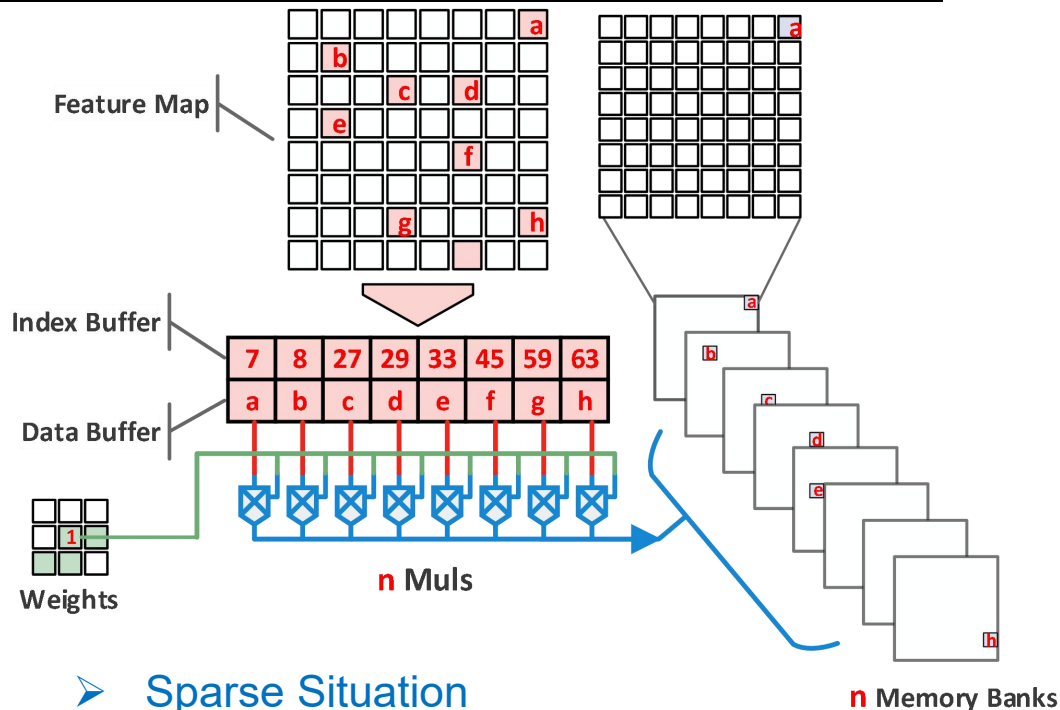


Challenge-2 Irregular Destination when Sparse



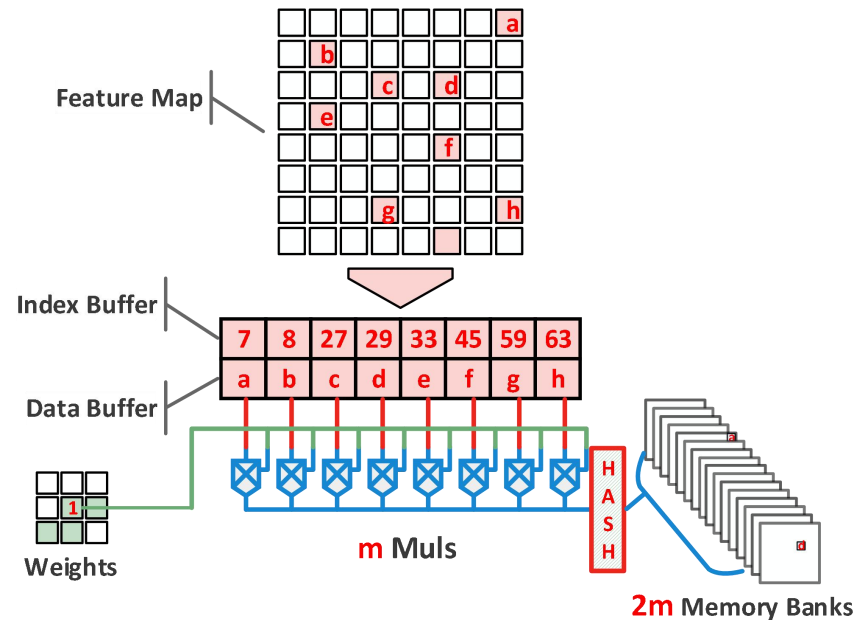
➤ Dense Situation

- ❑ Regular Data
- ❑ No Problem When MAC



➤ Sparse Situation

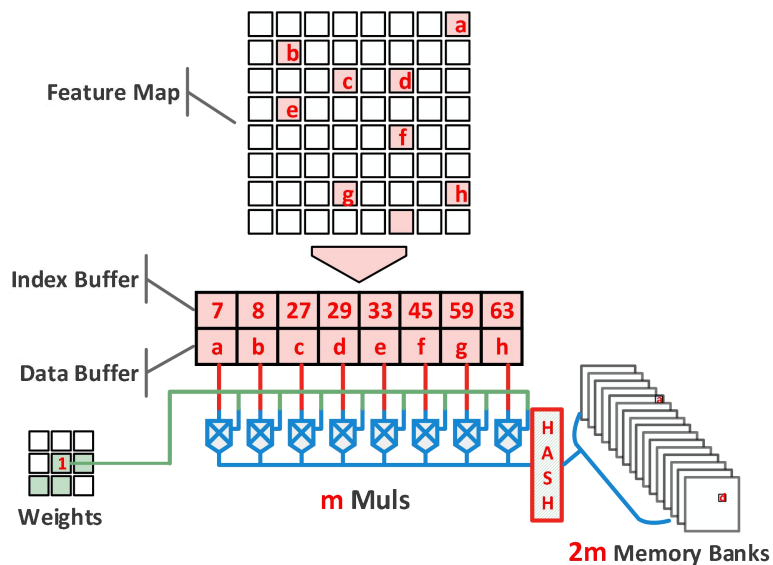
- ❑ Irregular Data
- ❑ Big Challenge When MAC



- Supporting Both Sparse Activ. And Weight
- SCNN [ISCA2017]

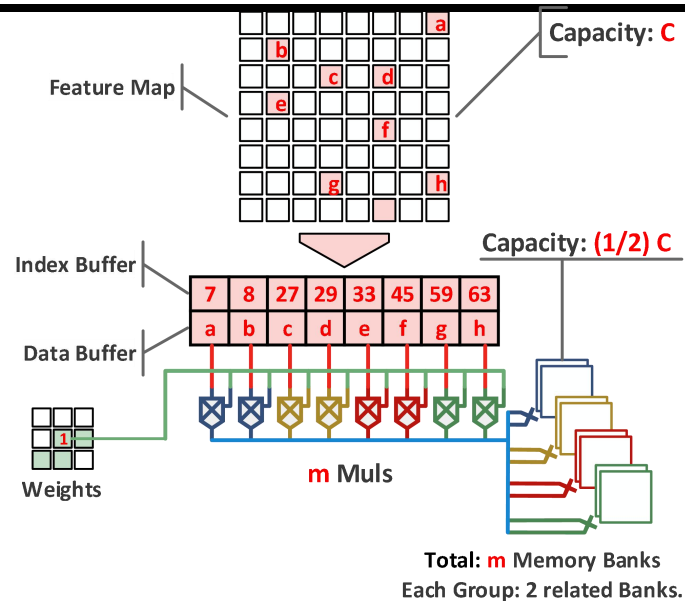
- ❑ Best Performance
- ❑ Low Area Efficiency

Comparison With Previous Work



➤ Hash PE (SCNN)

- ❑ 2M Memory Bank
- ❑ Fully Connection Hash Routing
- ❑ Complex Control Logic

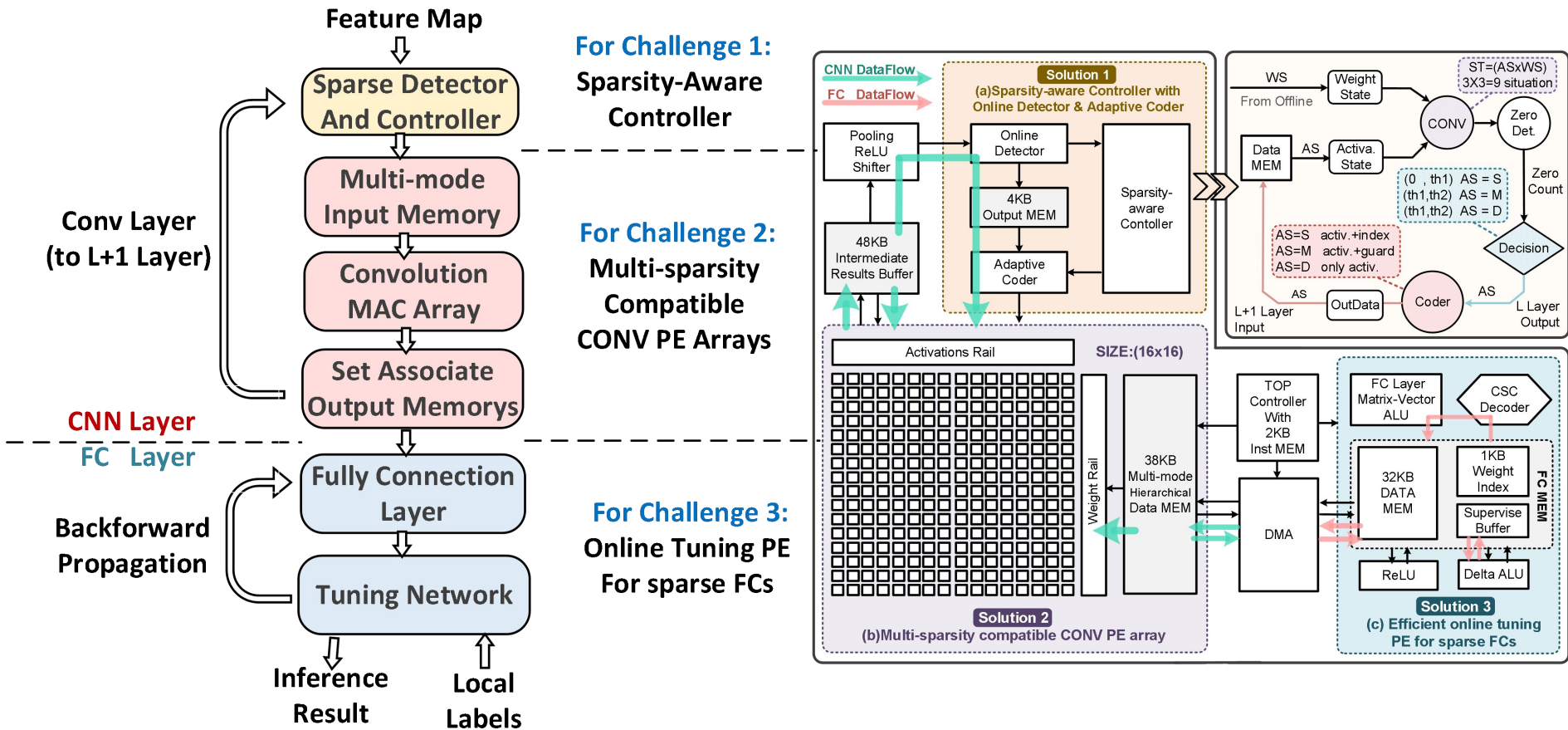


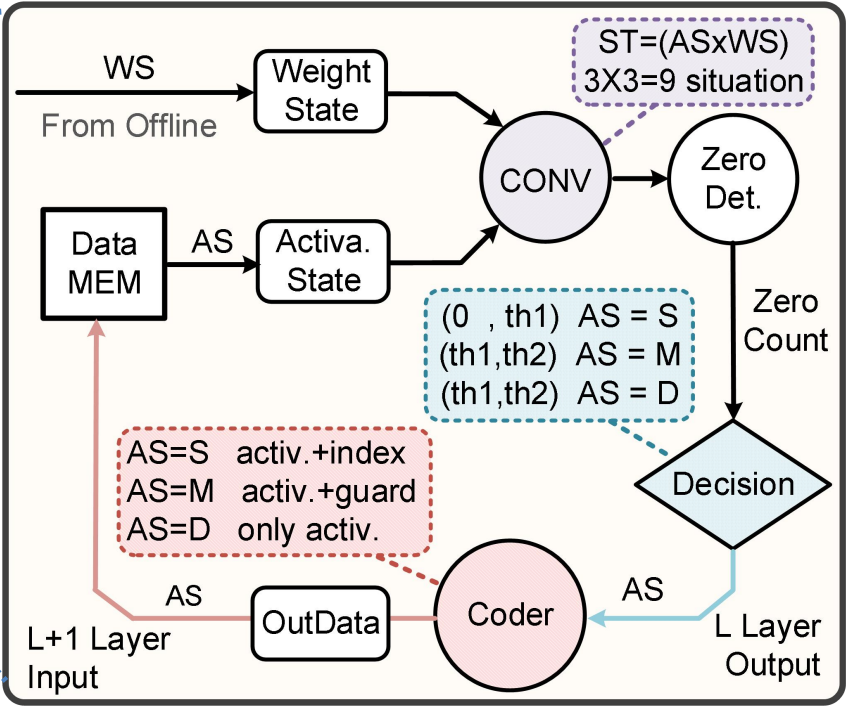
➤ This Work

- ❑ M Memory Bank
- ❑ 2x2 Cross Bar
- ❑ Simple Control Logic

**However
More Collision?**

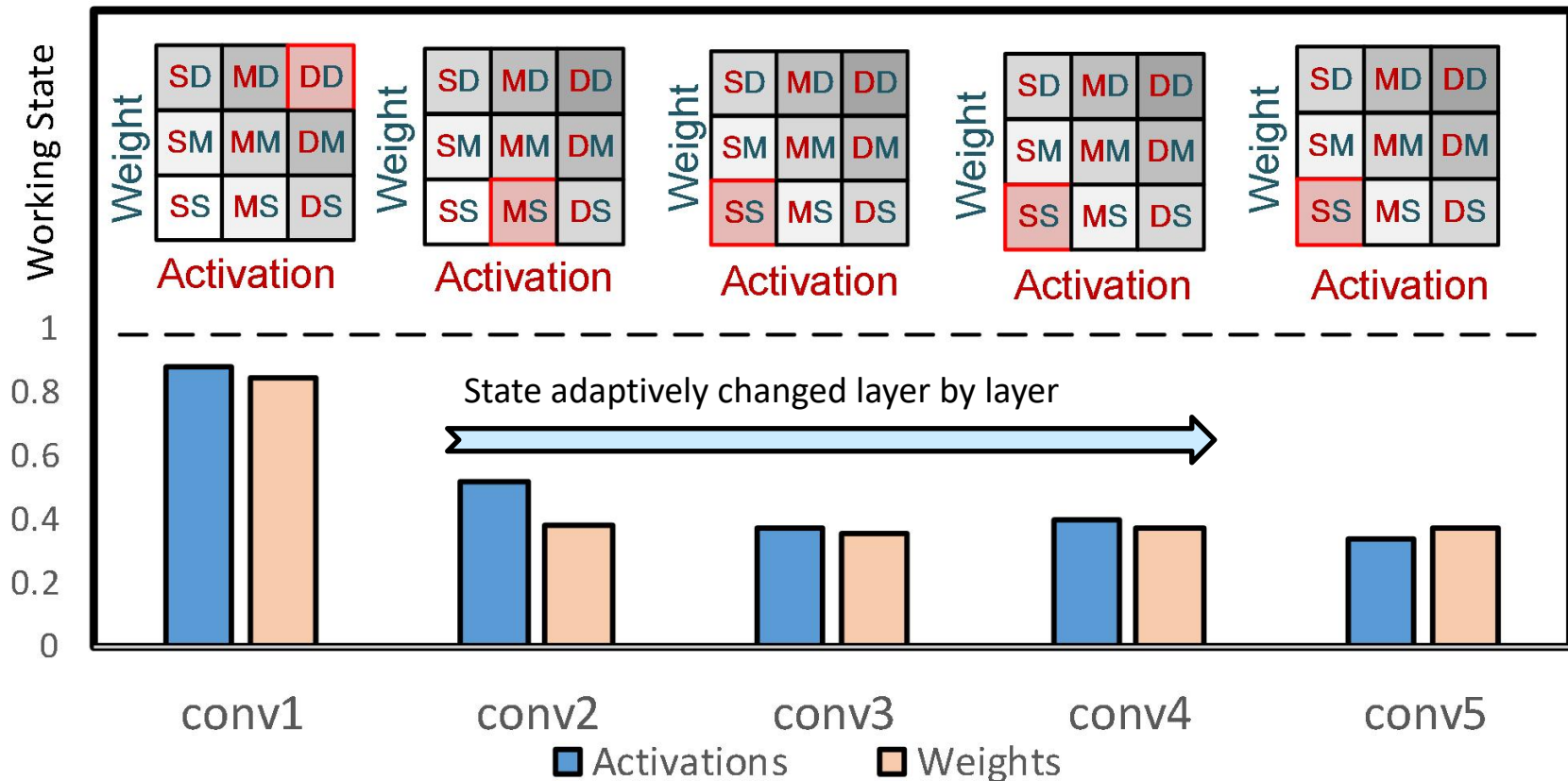
Chip Dataflow and Architecture Overview



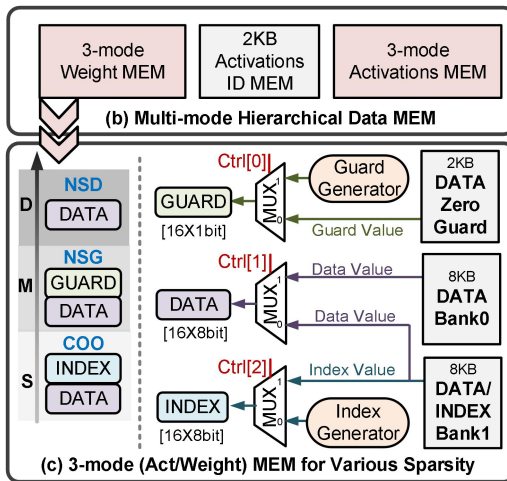
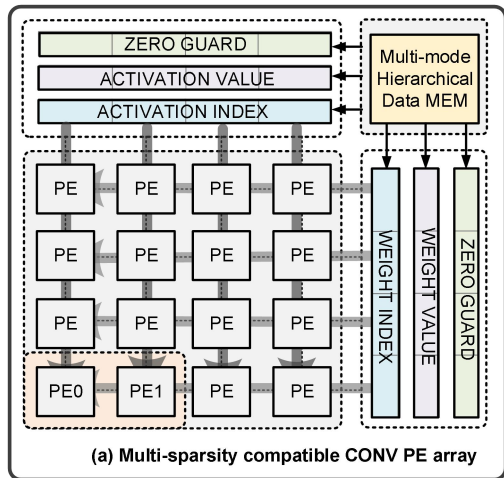


- Online + Offline Control Flow
- 3 Different Coder Types
- 9 Different Sparsity States

Chip States When Processing Alexnet



Multi-sparsity Compatible CONV PE Array

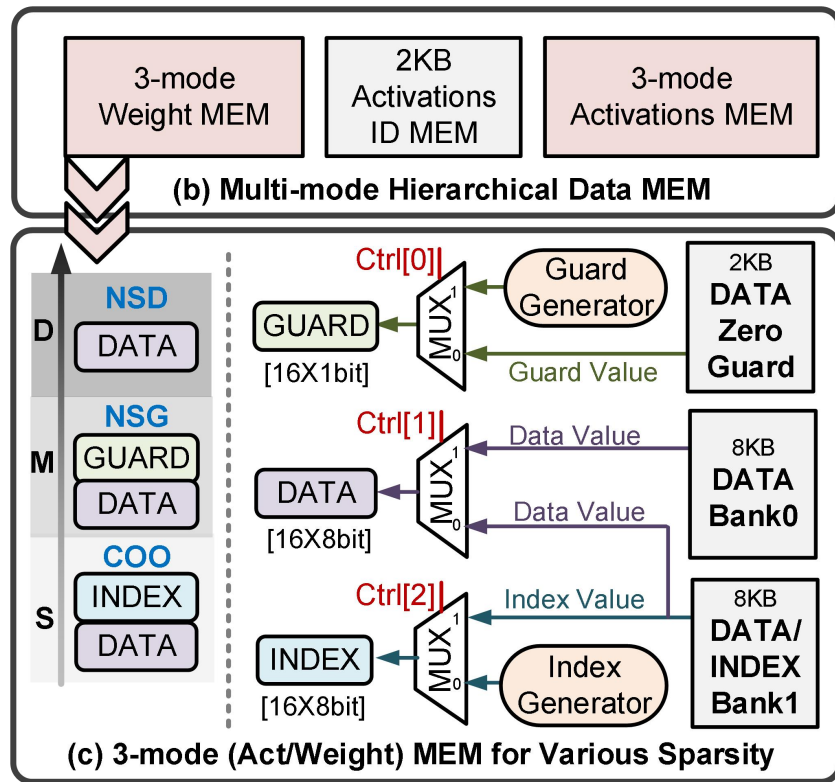


Convolution PE Array
Efficiency Data Reuse

Configurable Memory
Efficiency Input Storage

Set-Associate PEs
Efficiency Sparse CONV

Hierarchical Configurable Memory



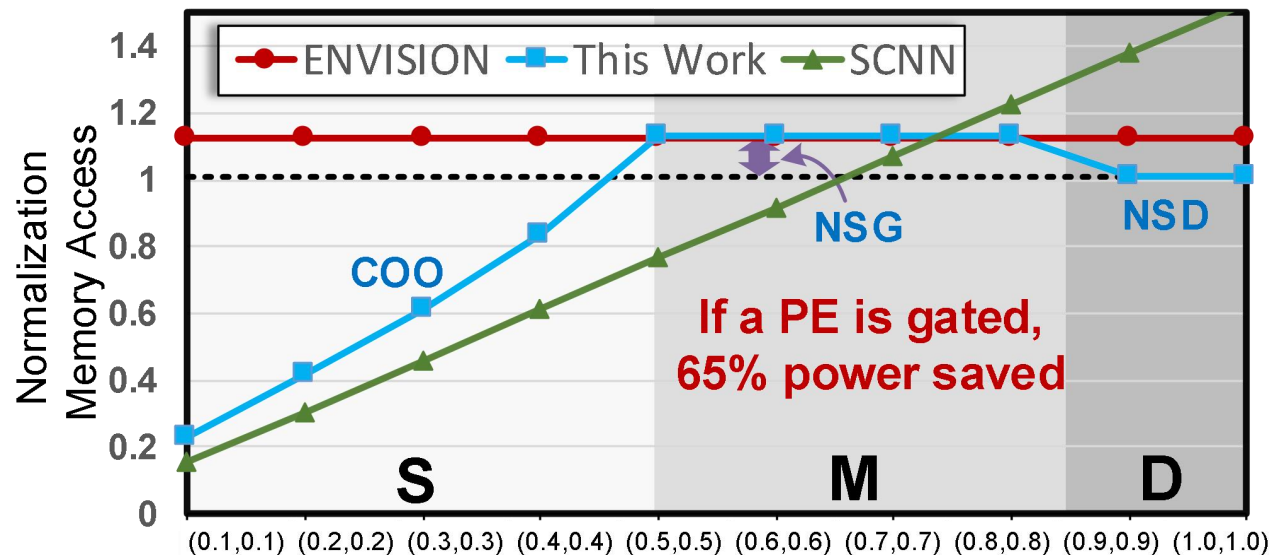
➤ Hierarchical Configurable Memory

- ❑ Two 3-mode Data Memory
- ❑ A ID Memory
- ❑ Configure by Inst + Online Controller

➤ 3-Mode Memory

- ❑ Non Zero Data + Data Index (COO)
- ❑ Data + Zero Guard
- ❑ Only Data

Storage Efficiency Comparison



➤ Comparison

□ ENVISION

□ SCNN

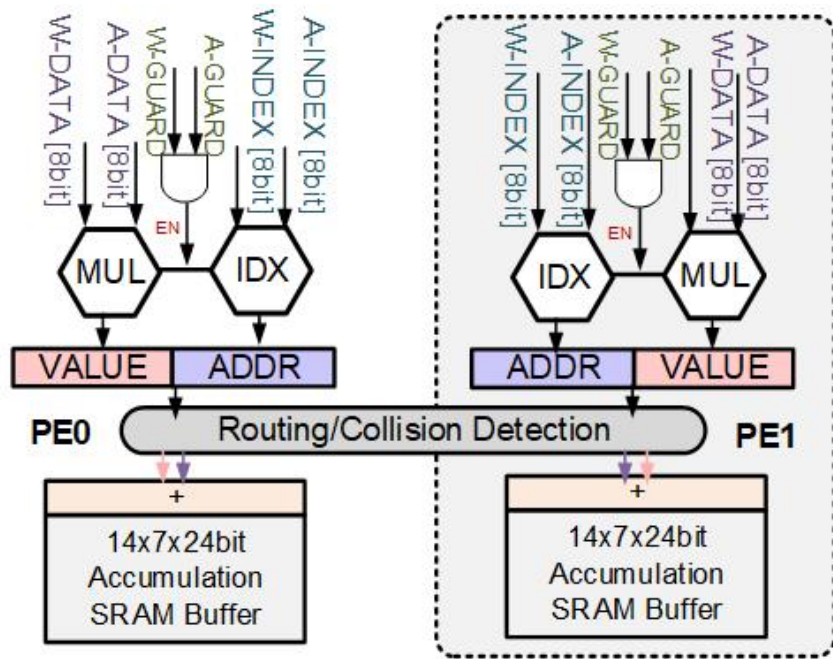
➤ Three States

□ Sparse (S)

□ Middle (M)

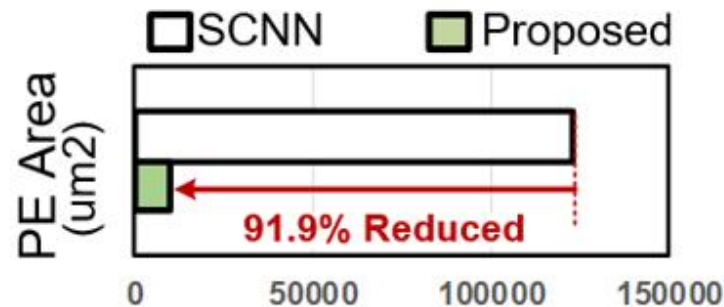
□ Dense (D)

Compatible Set-associative PEs



➤ Compatible Set-associative PEs

- ❑ Configurable Mode
- ❑ Set-associative Architecture
- ❑ Area Efficiency Solution



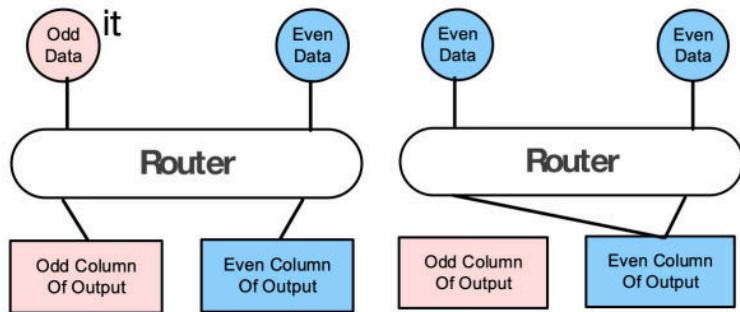
How to Reduce Collision

➤ Architecture Collision Design

- ❑ Only When Activ. Is in sparse state



- ❑ Even if Collision, 2 cycle can handle

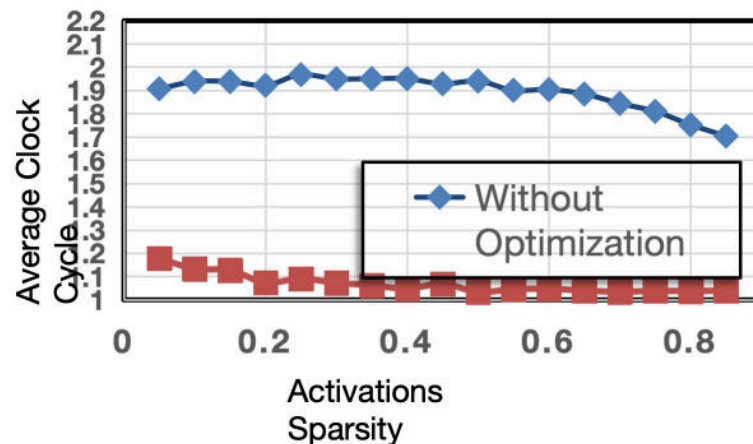
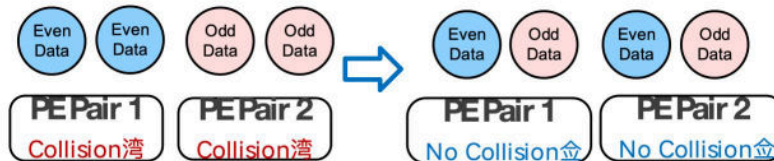


No Collision 😊

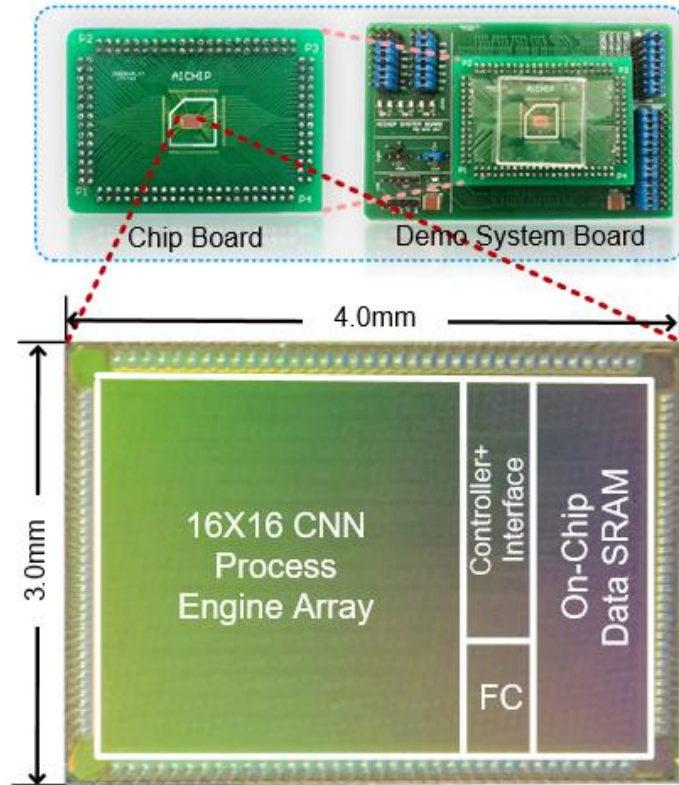
Collision 😞

➤ Software Data Arrangement Algorithm

- ❑ Arrange Input Feature Map

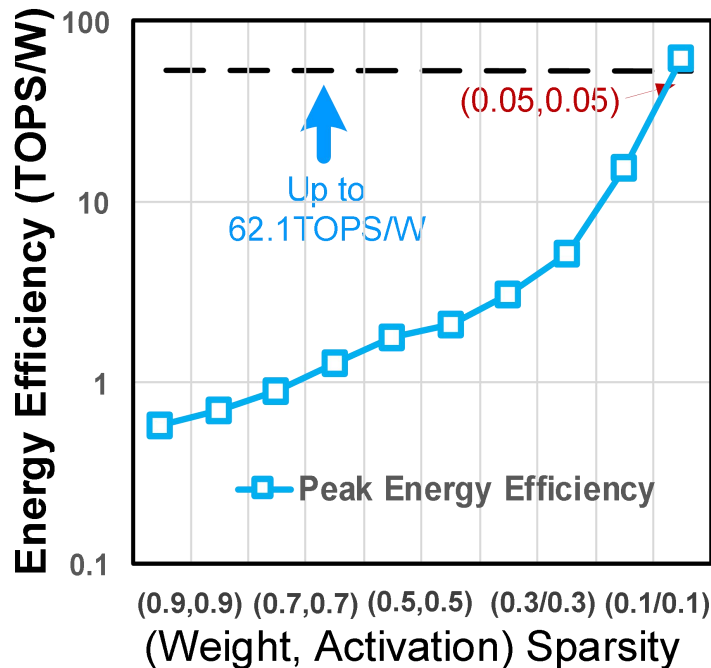


Chip Photograph and Summary

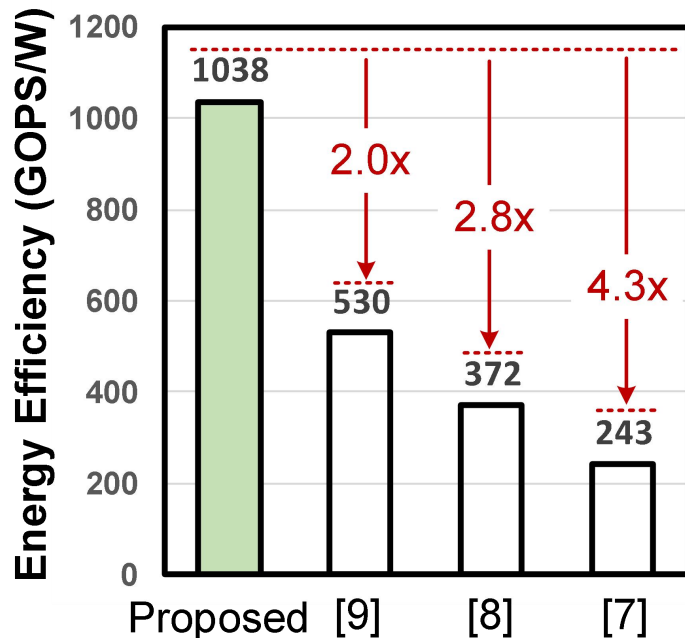


Technology	TSMC 65nm GP
Chip Area	4.0mmx3.0mm
Data Length	8 bit*
ACC Width	24 bit
Total SRAM	170 KB
Supply Voltage	0.67-1.0 V
Frequency	20-200 MHz
Peak Performance**	102-5638 GOPS
Energy Efficiency	0.411-62.1 TOPS/W
Power	20.5-248.4 mW
Layer Type	CNN+FC
Onchip Tuning	Supporting

Efficiency Comparison



Peak Energy Efficiency with Sparsity



*Under same inference accuracy on pruned Alexnet[2]

Energy Efficiency for Pruned Alexnet

Comparison Summary

	General					Function				Efficiency and Performance					
	Tape-out	Tech. (nm)	Core Area (mm ²)	Supply voltage (V)	Onchip SRAM	Sparsity Supporting			Online Tuning	power (mw)	Clock Freq. (MHz)	Data Length (bits)	MACs Num	Efficiency (TOPS/W)	
						Clock Skip	Power Gating	Dense Mode						Alexnet**	Peak
EIE[3]	NO	28	63.8	-	-	A+W	x	x	x	2360	1200	4	256	-	-
Cambricon-X[4]	NO	65	6.4	-	-	W	x	o	x	954	980	16	256	-	0.57
Cnvlutin[5]	NO	65	71.0	-	-	A	x	x	x	-	-	16	-	-	-
SCNN[6]	NO	16	7.90	-	1.2MB	A+W	x	x	x	-	1000	16	1024	-	-
EYERISS[7]	YES	65	12.3	0.82-1.17	108KB	x	A	x	x	235-332	250	16	168	0.243	0.31
VLSI2016[8]	YES	40	2.40	0.55-1.1	144KB	x	x	A+W	x	35-288	204	1-16	256	0.372	2.6
ENVISION[9]	YES	28	1.9	0.65-1.1	144KB	x	x	A+W	x	8-300	200	1-16	Nx256	0.53	10
This Work	YES	65	7.8	0.67-1.0	170KB	A+W	A+W	A+W	o	21-248	200	8*	256	1.038	62.1

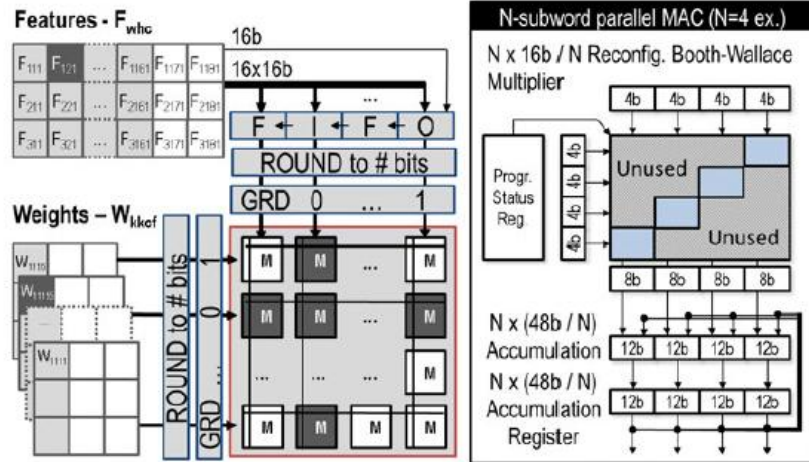
*To have enough accuracy, different layer has different quantization location as [10]

**Under same inference accuracy on pruned Alexnet from[2]

Adaptive Quantization

- **Word-parallel multi-bit multiplier:** reusing inactive arithmetic cells at reduced precision
 - Traditional multiplier: 1x1-16b words per cycle
 - Word-parallel: 2x1-8b or 4x1-4b ($N \times 1-16b/N$) words per cycle
- **Lower processor's frequency and voltage at constant data throughput**

ENVISION@ISSCC17

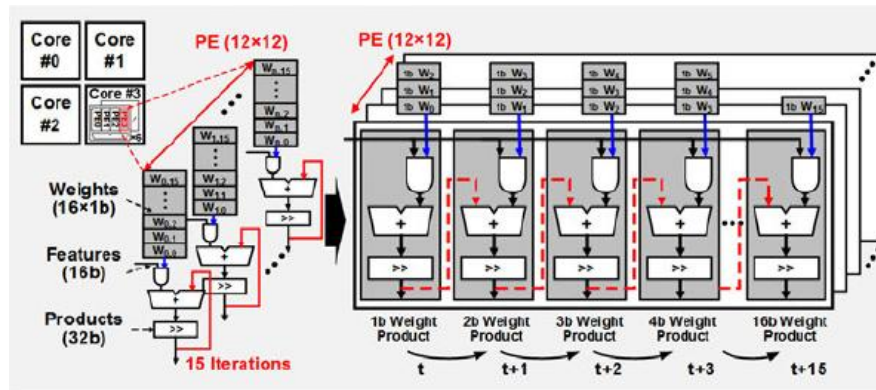


	[2] GLSVLSI '15	[3] ISSCC '16	[4] VLSI '16	This work $N = 1, 2$ or 4
Technology	65nm CMOS	65nm LP CMOS	40nm LP CMOS	28nm UTBB FD-SOI
Nominal Frequency [MHz]	500	200	200	200
Supply @ f_{max} [V]	1.2	1	1.1	1
Peak performance [GOPS]	Fixed 196	Fixed 67	Fixed 102	Dynamic $N \times 102$
Active Area [mm ²]	1.31	12.25	2.4	1.87
# of MACs	-	168	256	Dynamic $N \times 256$
Gate Count [NAND 2]	0.9M	1.852M	1.6M	1.95M
On-Chip SRAM [kB]	43	184.5	144	144
# layers, # filters [-]	All	All	All	All
Filter sizes [-]	<7x7	1-1024	All	All
Precision [bits]	Fixed 12	Fixed 16	Dynamic 1-16	Dynamic $N \times 1-16 / N$
AlexNet Conv-layers [mW]	-	278 @ 34.7fps	55-95, 76 avg. @ 47fps	20-62, 44 avg. @ 47fps
VGG Conv-layers [mW]	-	-	-	19-35, 26 avg. @ 1.67fps
Dynamic power range @ $GOPS_{avg}$ [mW]	510 (1x) @ 145 GOPS	235-332 (1.5x) @ 51-56 GOPS	35-288 (8.2x) @ 80 GOPS	7.5-300 (40x) @ 76 GOPS
Min. efficiency [TOPS/W]	0.44	0.15	0.27	0.26
Max. efficiency [TOPS/W]	0.8	0.35	2.6	10

Adaptive Quantization

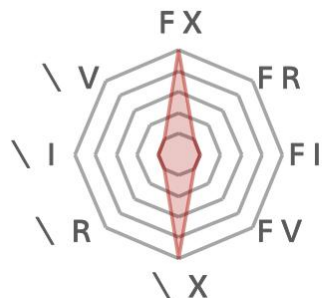
- Unified neural processing unit (UNPU) for convolution, fully-connected layers and recurrent layers.
 - Lookup-table-based bit-serial processing element: flexible bit precision. (Fully-variable weight precision from 1b to 16b)
 - Efficient input feature reuse under low weight bit-precision

UNPU@ISSCC18

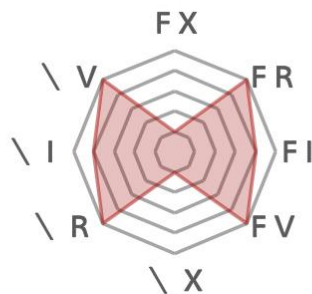


	ISSCC2017 [1]		ISSCC2017 [2]	S.VLSI2017 [5]		S.VLSI2017 [6]	This Work
Purpose	CLs, RLs, FCLs		CLs	CLs, RLs, FCLs		RLs, FCLs	CLs, RLs, FCLs
Process [nm]	65		28	65		65	65
Die Area [mm ²]	16		1.87	19.36		12	16
Supply Voltage [V]	0.77-1.1		0.605-1.05	0.67-1.2		0.55-1.0	0.63-1.1
Max. Freq. [MHz]	200		200	200		400	200
SRAM [KB]	290		144	348		153	256
PE Bit-precision [bit]	4, 8, 16		4, 8, 16	4, 8, 16		1, 2 (Binarized Activation)	1-16 (Continuous Activation)
Peak Performance [GOP/s]	1200(4b) 300(16b)		408(4b) 76(16b)	409.6 (4b)		1380(1b)	7372(1b), 1382(4b), 691.2(8b), 345.6(16b)
Energy Efficiency [TOPS/W]	@50MHz 0.77V	8.1 (4b)	@50MHz 0.63V	10.0 ¹¹ (4b)	@10MHz 0.67V	5.09 (4b)	6.0 (1bx1b)
		2.1 (16b)	@100MHz 0.70V	0.53 (16b)	@200MHz 1.2V	1.06	2.3 (1b1b)
Power Consumption [mW]	@50MHz 0.77V	34.6	@50MHz 0.63V	7.5	@10MHz 0.67V	4	50
		279	@200MHz 1.05V	300	@200MHz 1.2V	447	600
	@200MHz 1.1V				@400MHz 1.0V		@5MHz 0.63V
							@200MHz 1.1V
							297

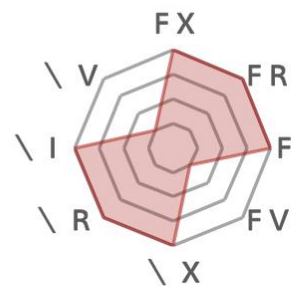
Complete Sparse Network Coverage



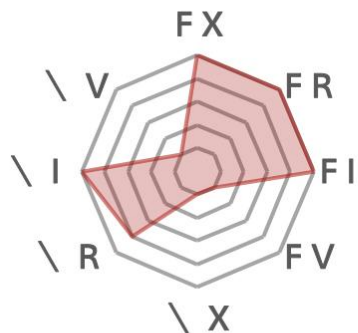
SCNN



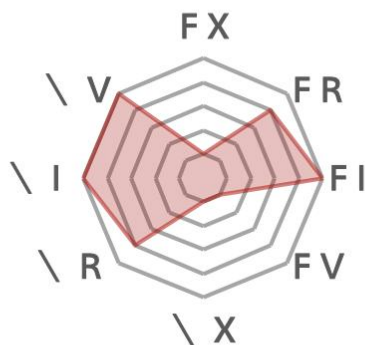
EVISION



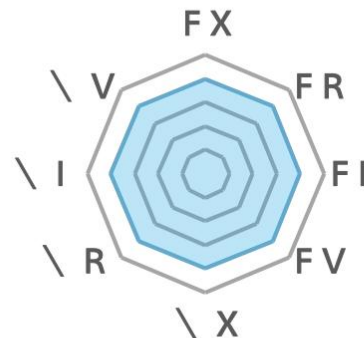
STICKER



EYERISS



UNPU



Design Target

Two Data Type

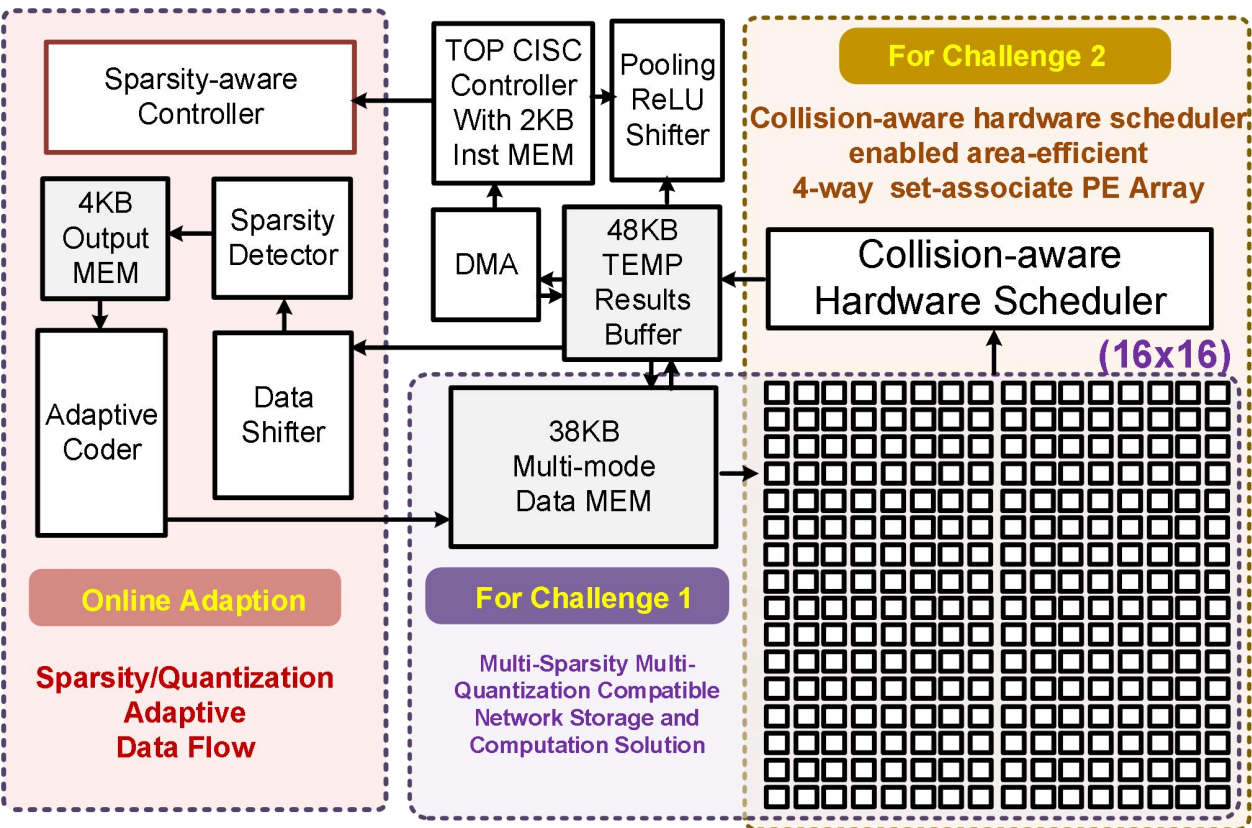
A: Activation
W: Weight

S: Sparse
M: Medium
D: Dense

Q: Multi-bit
Quantization

Design Target
Complete Network
Coverage

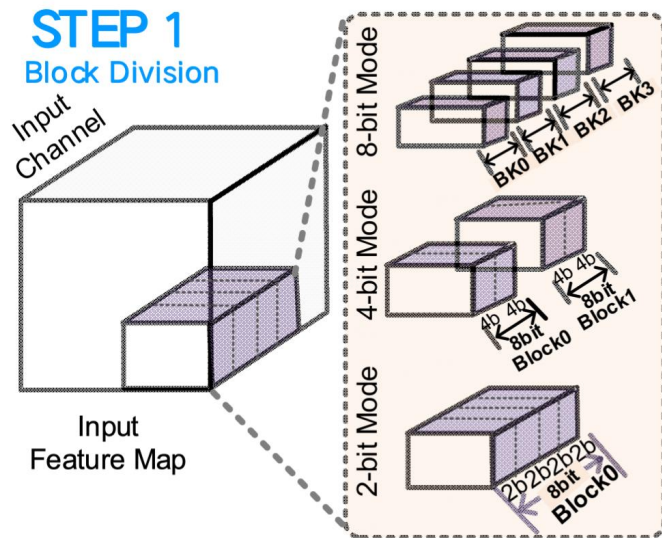
STICKER-II Architecture



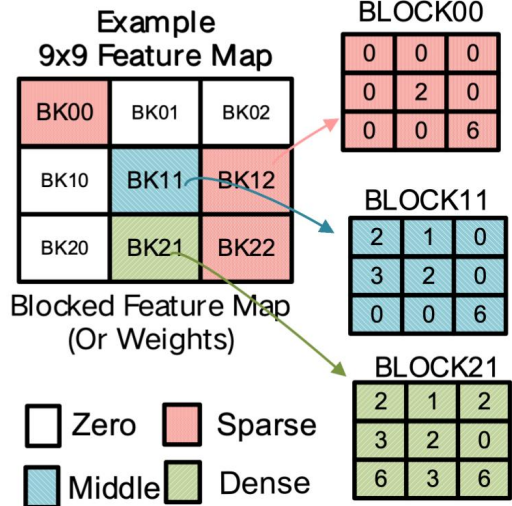
➤ Architecture Overview

- ❑ Adaptive Sparsity/Quant.
- ❑ Sparsity/Quantization
- ❑ Collision-Aware Controller

Multi-Sparsity Multi-Bit Storage Format



STEP 2 Block Sparsity Decision



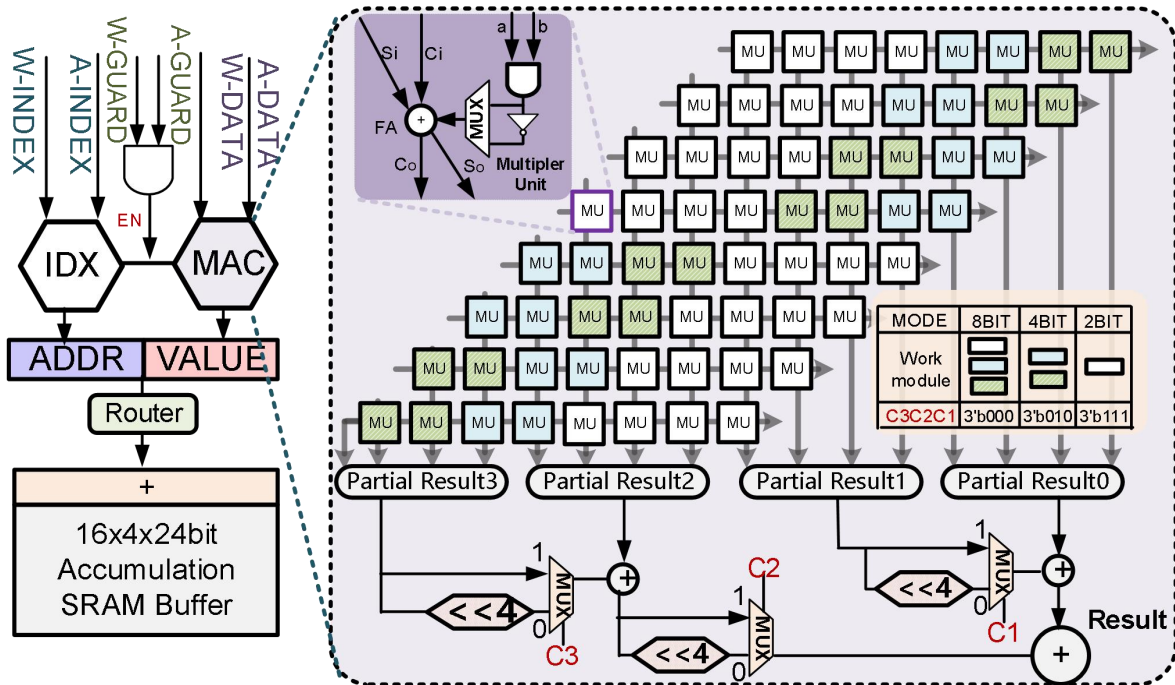
➤ MSMB Format

- ❑ Shorter Quantization → Less Blocks
- ❑ Different Channel Data in one Block Share an Index
- ❑ Blocks are Classified into 4 Different Type

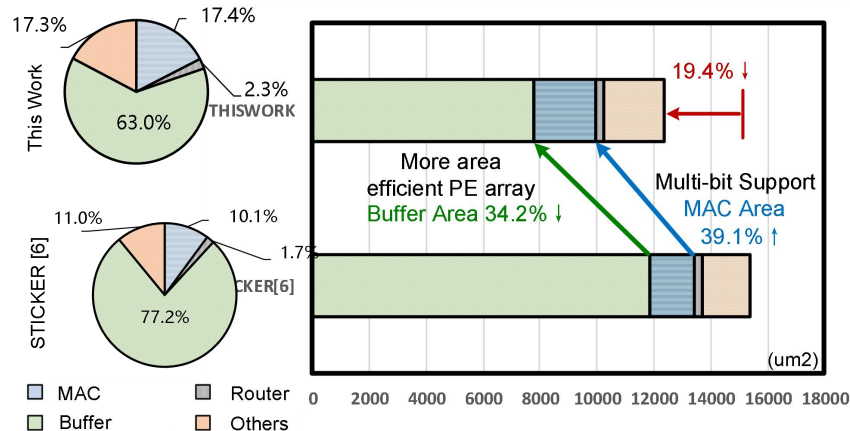
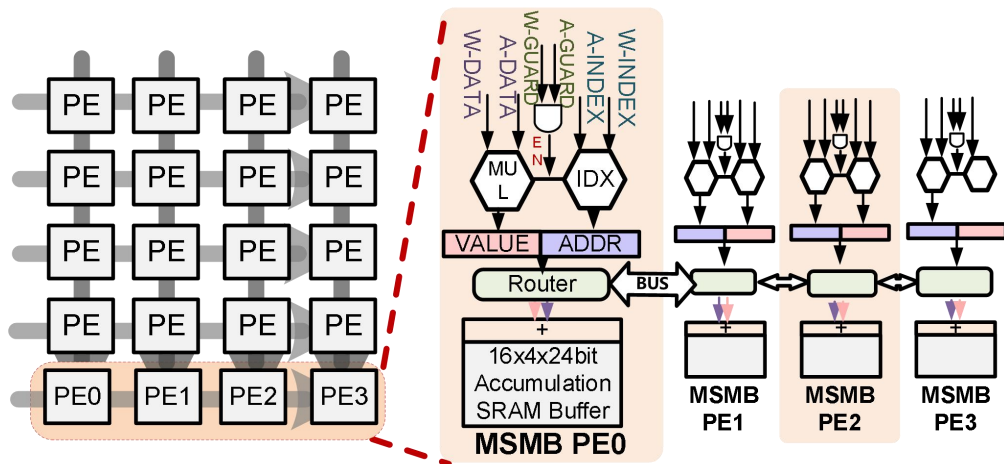
MSMB Supporting MAC

MSMB MAC

- ❑ Power Off Unused MU
- ❑ 8BIT/4BIT/2BIT
- ❑ Directly Get Channels Sums
- ❑ First time, Partial Sums
- ❑ Write into ACC Buffers
- ❑ Second time, Final Sums



Set Associate PE Array



➤ Set Associate PE Array

- ❑ Multi-Sparsity Supporting
- ❑ N PE Share Buffer Set

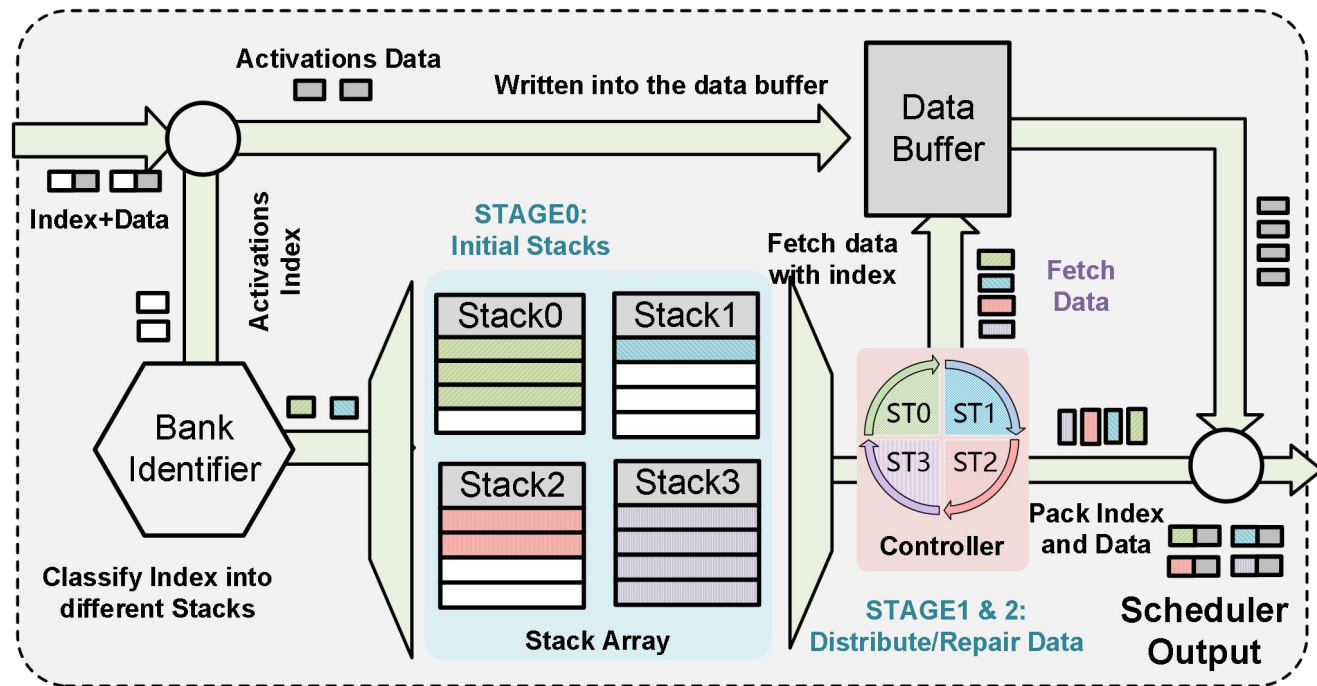
➤ 4 Way Set Associate PE Implementation

- ❑ Buffer Occupies Most Area Budget
- ❑ **19.4% Smaller** Area with Multi-bit MAC

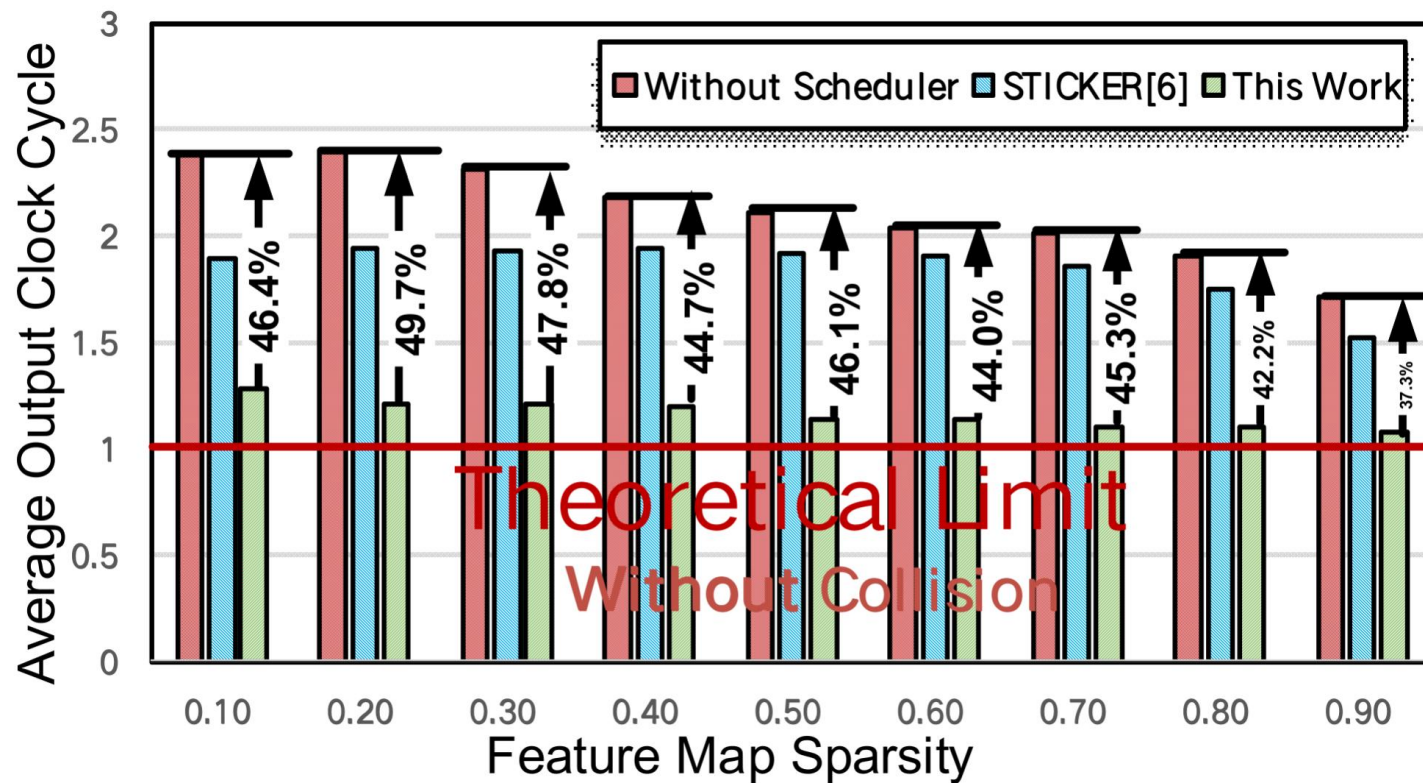
Collision-Free Scheduler

Hardware Architecture

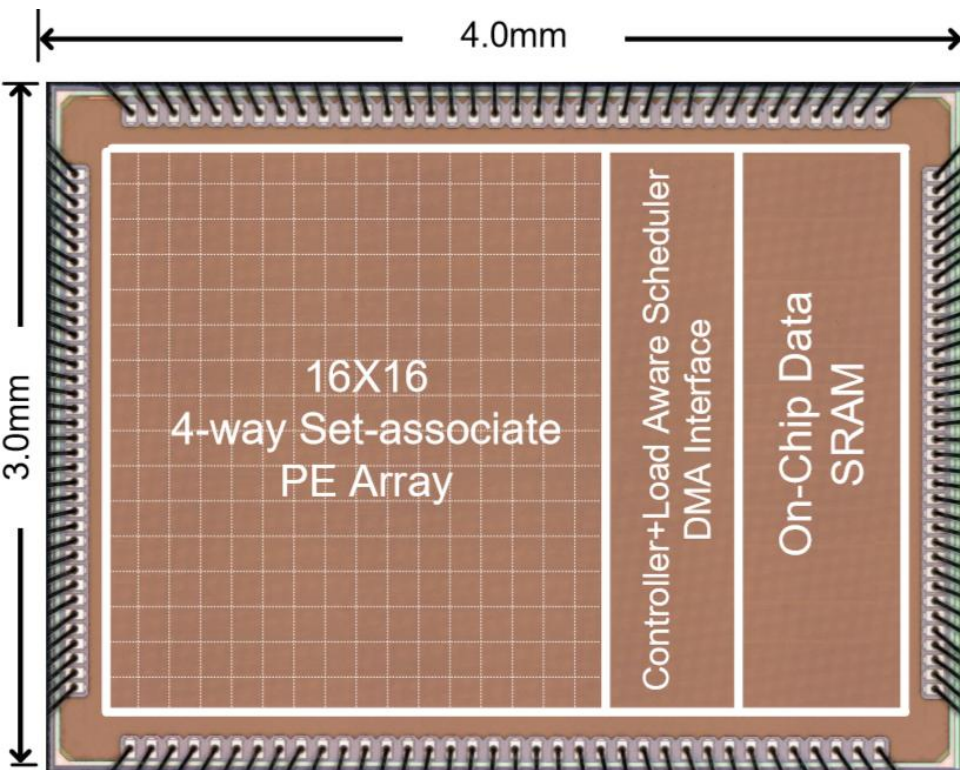
- ❑ Schedule Layer Results
- ❑ Index/Data Division
- ❑ 4 Stacks Distribution
- ❑ Index/Data Combination
- ❑ Overlap Time Overhead



Collision and Scheduler Result



STICKER-II Chip Micrograph



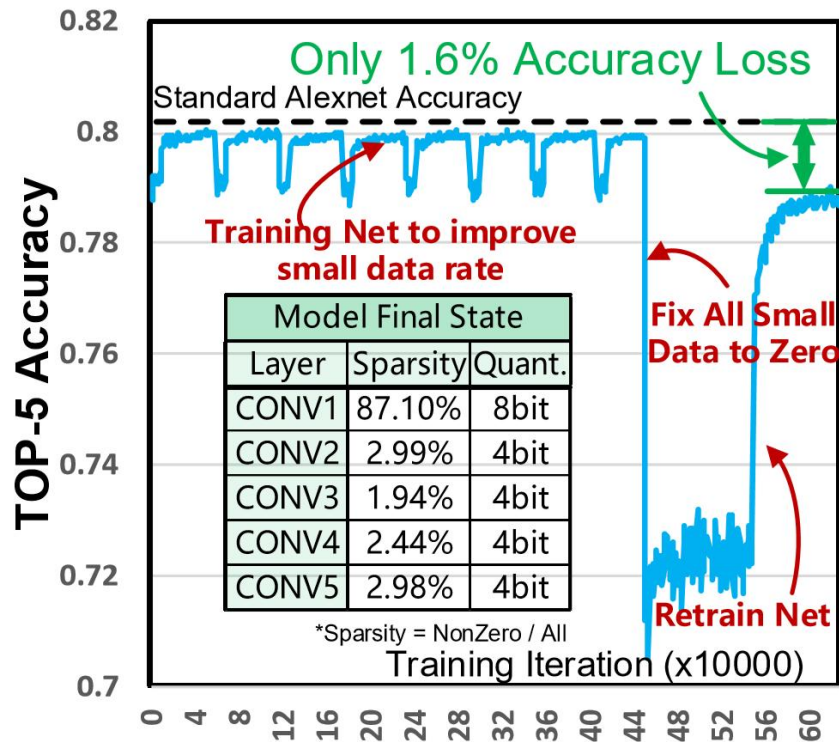
Chip Photograph

Technology	TSMC 65nm GP
Chip Area	4.0mmx3.0mm
Data Length	2-8bit
Total SRAM	170 KB
Supply Voltage	0.52-1.13V
Core Frequency	25M-200M
Power Consumption	15.1- 318.6 mW
Energy Efficiency*	0.32 - 246.0 TOPS/W

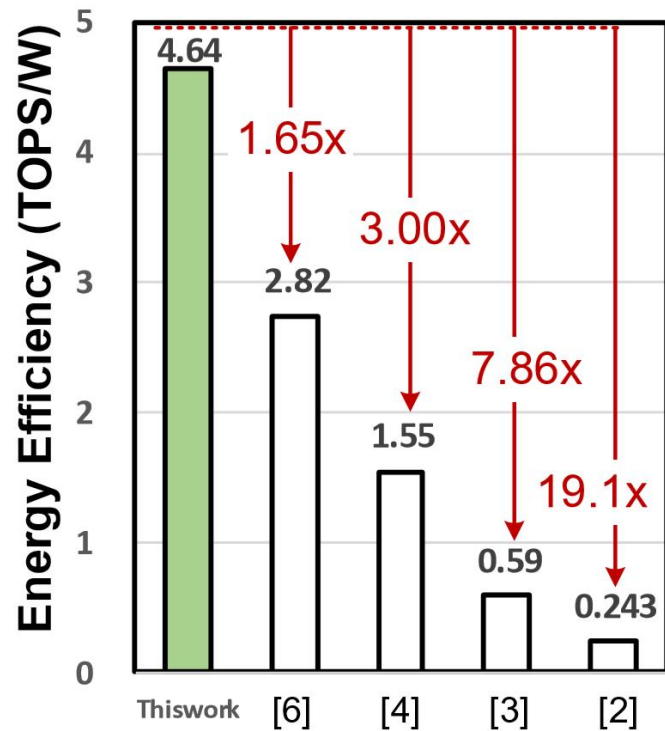
* Efficiency Varies with different
Voltage/Frequency/Sparsity/Quantization

Chip Summary

Real World Network Example



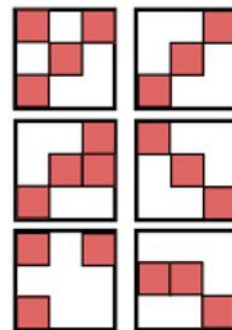
(a) Alexnet Pruning Process



(b) Alexnet Energy Efficiency

Two Ways to Utilize Sparsity

- Unstructured sparsity utilization
 - Sticker in VLSI 2018
 - Sticker-II in A-SSCC 2019



- Structured sparsity utilization
 - Sticker-T in ISSCC 2019

-1.39	0.06	1.56	0.36
0.36	-1.39	0.06	1.56
1.56	0.36	-1.39	0.06
0.06	1.56	0.36	-1.39

CirCNN Mechanism

[CirCNN, Micro2017]

$$\begin{bmatrix}
 -1.39 & 0.06 & 1.56 & 0.36 \\
 0.36 & -1.39 & 0.06 & 1.56 \\
 1.56 & 0.36 & -1.39 & 0.06 \\
 0.06 & 1.56 & 0.36 & -1.39
 \end{bmatrix}
 \times
 \begin{bmatrix}
 0.36 \\
 0.43 \\
 -0.12 \\
 3.42
 \end{bmatrix}
 =
 \begin{bmatrix}
 0.57 \\
 4.86 \\
 1.09 \\
 -4.10
 \end{bmatrix}$$

W_{ij} x y

Block-circulant

Block-circulant FC/RNN

$$\begin{array}{c}
 \begin{array}{ccc}
 \text{3D Tensor } W & * & \text{3D Tensor } F \\
 \downarrow r^2 C & & \downarrow P \\
 \text{Matrix } X & \times & \text{Matrix } F \\
 \downarrow (H-r+1)(W-r+1) & & \downarrow P \\
 \text{Matrix } Y & & \text{Matrix } Y
 \end{array}
 \end{array}$$

X F Y

Block-circulant

Block-circulant CNN

Unified

Matrix-vector multiplication



Matrix-matrix multiplication

CirCNN Training Flow

[CirCNN, Micro2017]

-1.39	0.06	1.56	0.36
0.36	-1.39	0.06	1.56
1.56	0.36	-1.39	0.06
0.06	1.56	0.36	-1.39

Circulant matrix

X	X	X	X
X	X	X	X
X	X	X	X
X	X	X	X
X	X	X	X
X	X	X	X
X	X	X	X
X	X	X	X

Unstructured weight
(32 parameters)

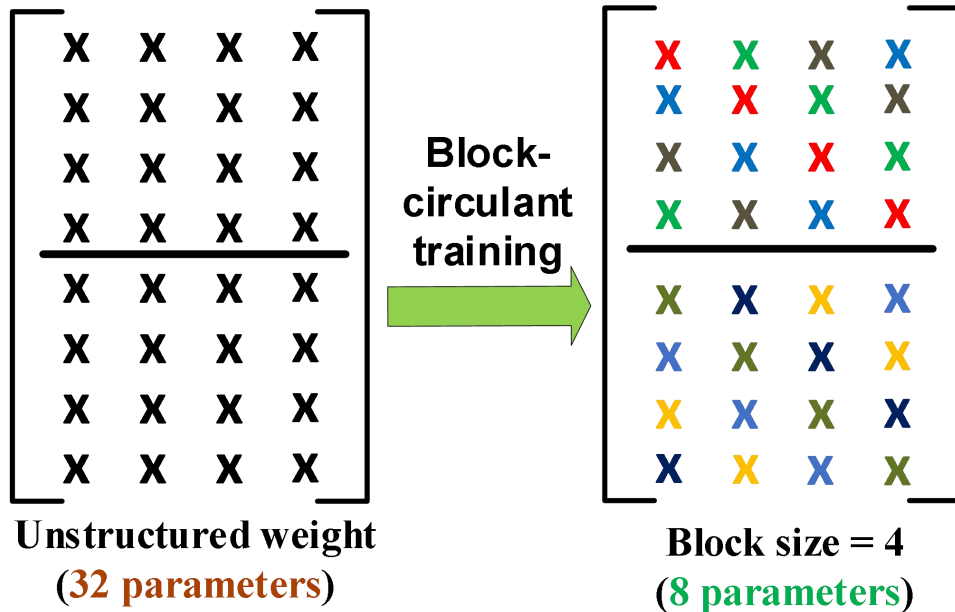
Block-
circulant
training

X	X	X	X
X	X	X	X
X	X	X	X
X	X	X	X
X	X	X	X
X	X	X	X
X	X	X	X
X	X	X	X

Block size = 4
(8 parameters)

CirCNN Training Flow

[CirCNN, Micro2017]



Input: $\frac{\partial L}{\partial \mathbf{a}}$, $\mathbf{w}_{ij}$'s, $\mathbf{x}$, p , q , k

Output: $\frac{\partial L}{\partial \mathbf{w}_{ij}}$'s, $\frac{\partial L}{\partial \mathbf{x}}$

Initialize $\frac{\partial L}{\partial \mathbf{w}_{ij}}$'s and $\frac{\partial L}{\partial \mathbf{x}}$ with zeros.

for $i \leftarrow 1$ until p do

for $j \leftarrow 1$ until q do

$\frac{\partial L}{\partial \mathbf{w}_{ij}} \leftarrow \text{IFFT}(\text{FFT}(\frac{\partial L}{\partial \mathbf{a}_i}) \circ \text{FFT}(\mathbf{x}'_j))$

$\frac{\partial L}{\partial \mathbf{x}_j} \leftarrow \frac{\partial L}{\partial \mathbf{x}_j} + \text{IFFT}(\text{FFT}(\frac{\partial L}{\partial \mathbf{a}_i}) \circ \text{FFT}(\mathbf{w}_{ij}))$

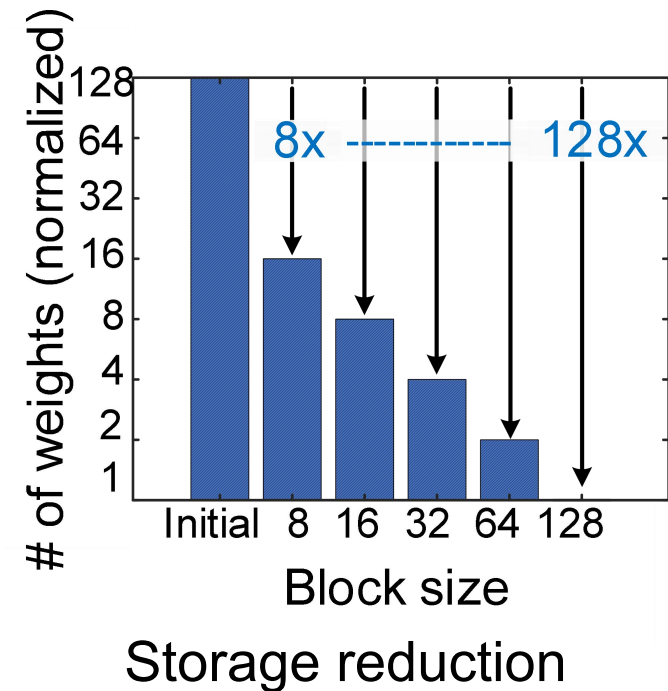
end

end

return $\frac{\partial L}{\partial \mathbf{w}_{ij}}$'s, $\frac{\partial L}{\partial \mathbf{x}}$

Circulant back-propagation

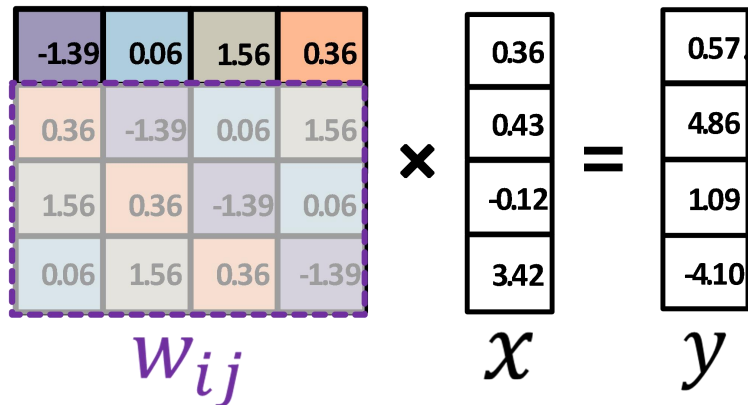
Storage & Accuracy After Training



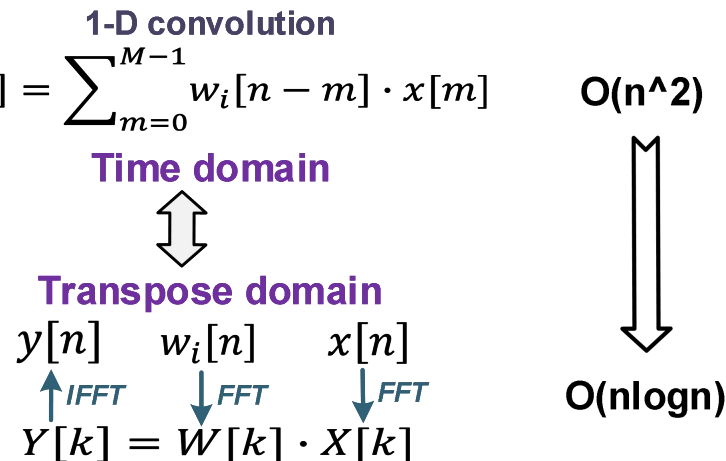
Model	Tiny YOLO		LSTM		FC	
Dataset	DataDJI		TIMIT		MNIST	
Layer type	CNN		RNN (LSTM)		FC	
Block size	16	32	16	32	64	128
MAC bit-precision	5	5	4	4	2	2
Accuracy loss	1.30%	3%	1.01%	1.93%	1.00%	1.50%

Transform-Domain Acceleration

Unified CNN/FC/RNN flow

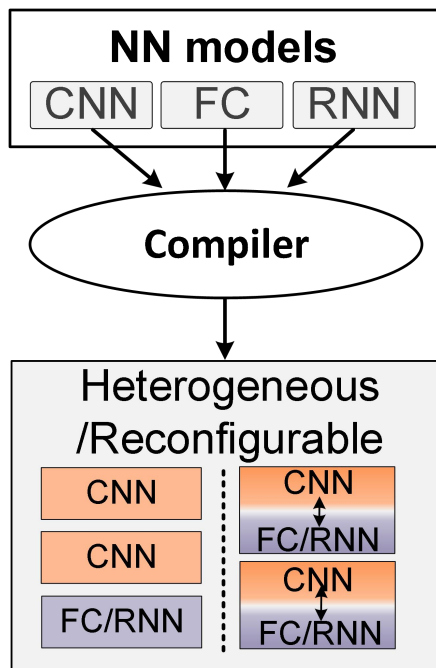


Transform-domain acceleration

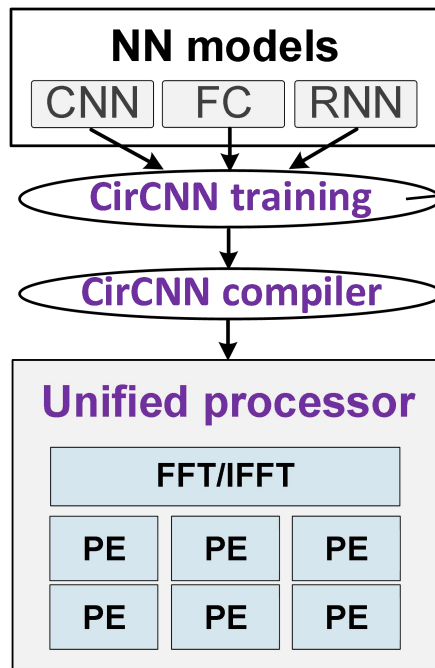
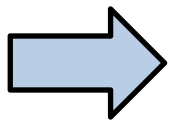


n (block-size)	8	16	32	64	128
$O(n^2)/O(n \log n)$	2.7x	4.0x	6.4x	10.7x	18.3x

CirCNN: Block-Circulant NN Workflow



Traditional workflow

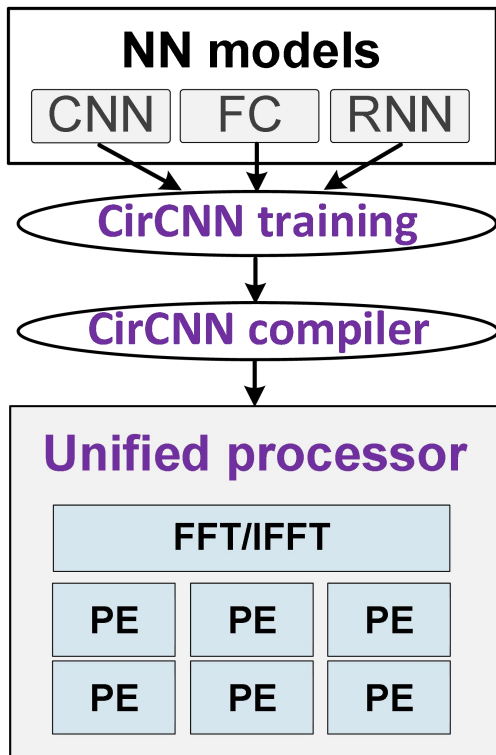


CirCNN workflow

Unified workflow

- Storage reduction
- Small accuracy loss
- T-D acceleration

CirCNN: Block-Circulant NN

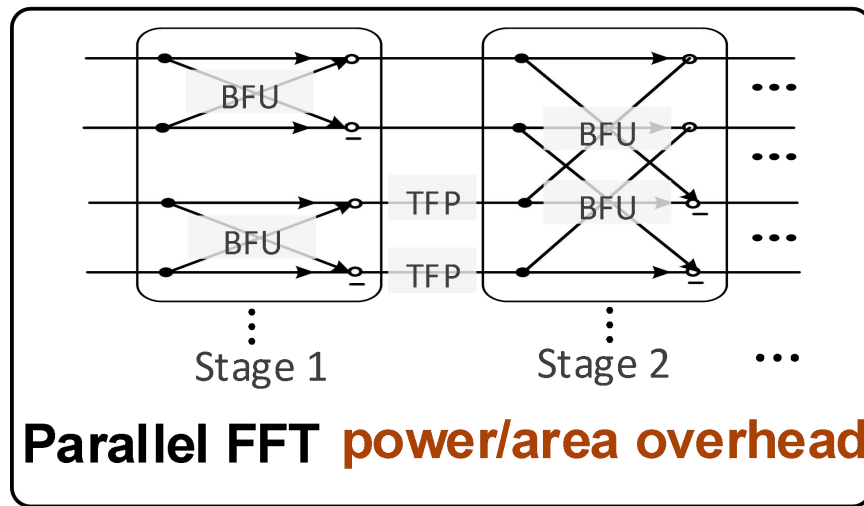
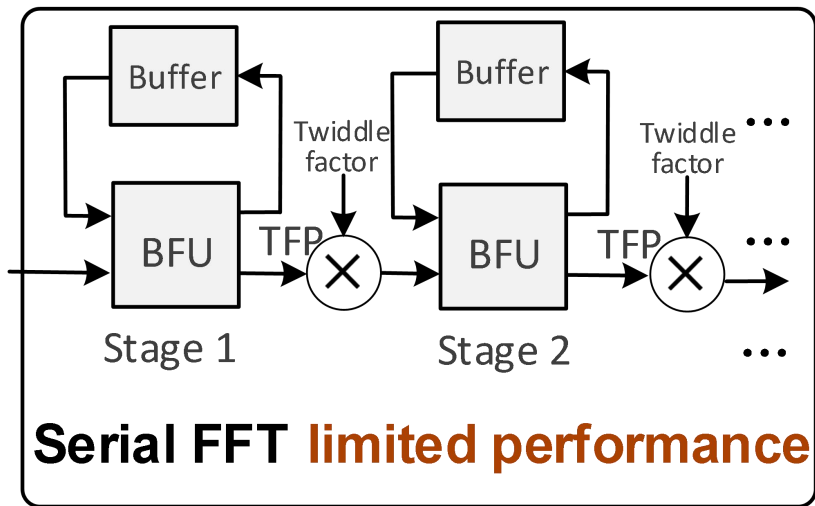


- **Unified workflow**
- **Storage reduction**
- **Small accuracy loss**
- **T-D acceleration**

Main operations in CirCNN processor

Fast Fourier Transformation (FFT)
Element-wise production

Challenge 1: FFT Overhead

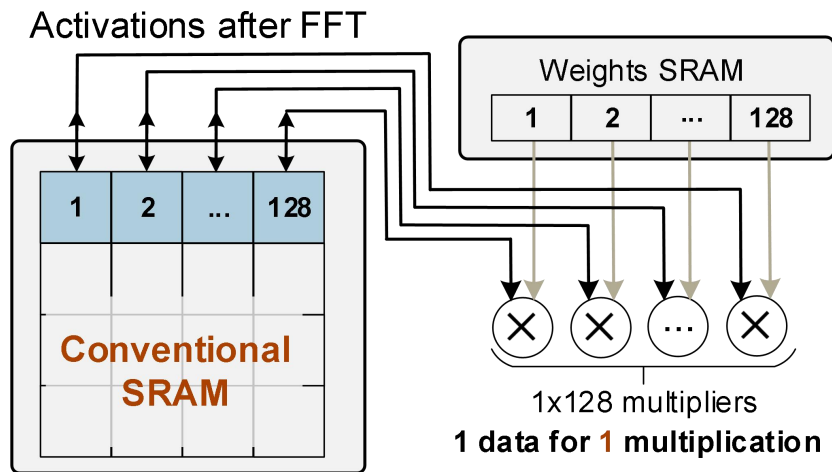


Global-parallel bit-serial FFT

Challenge 2: Data Reuse Problem

$$Y[k] = W[k] \cdot X[k]$$

Element-wise multiplication



**Conventional SRAM based
MAC array**

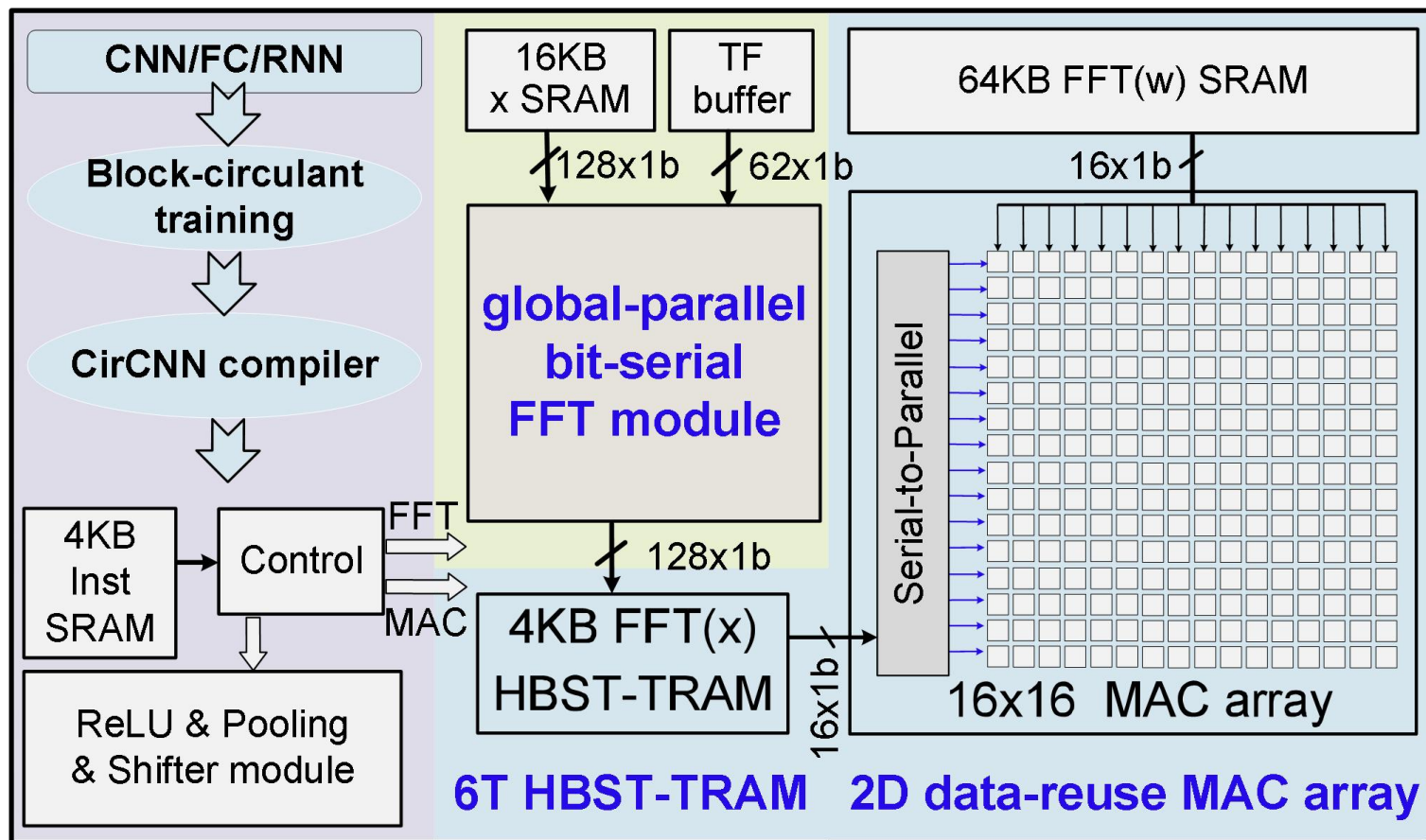
No data reuse

High bandwidth

High memory access

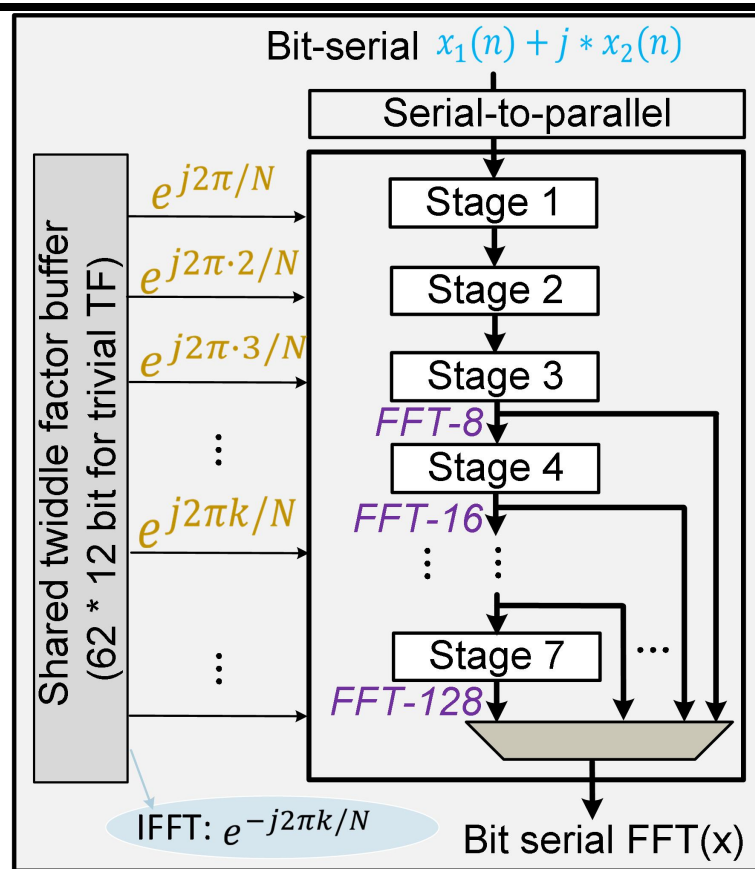
**HBST-TRAM based
2-D data-reuse array**

Overall Architecture of STICKER-T

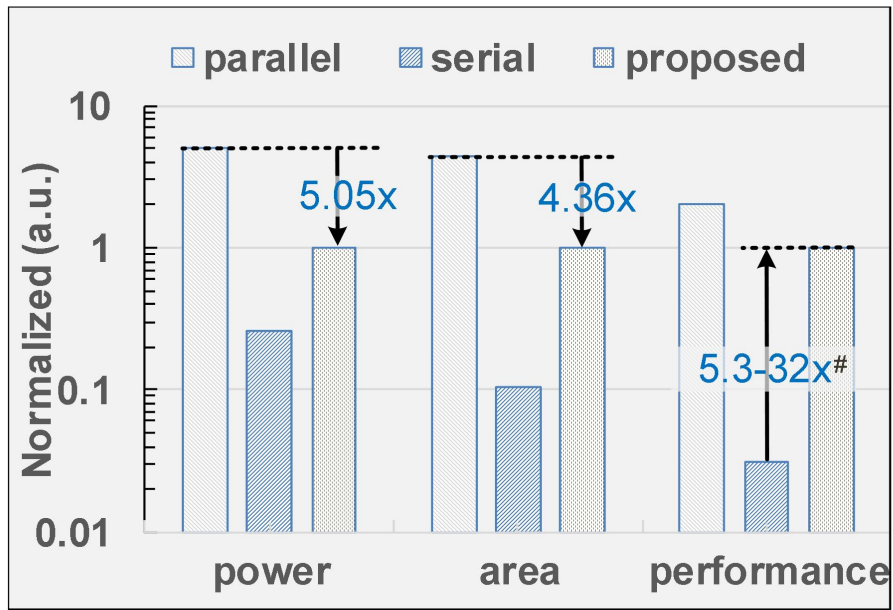


Global-Parallel Bit-Serial FFT

- Shared twiddle-factor (TF) buffer
- Skip non-trivial TFs
- Real activation feature

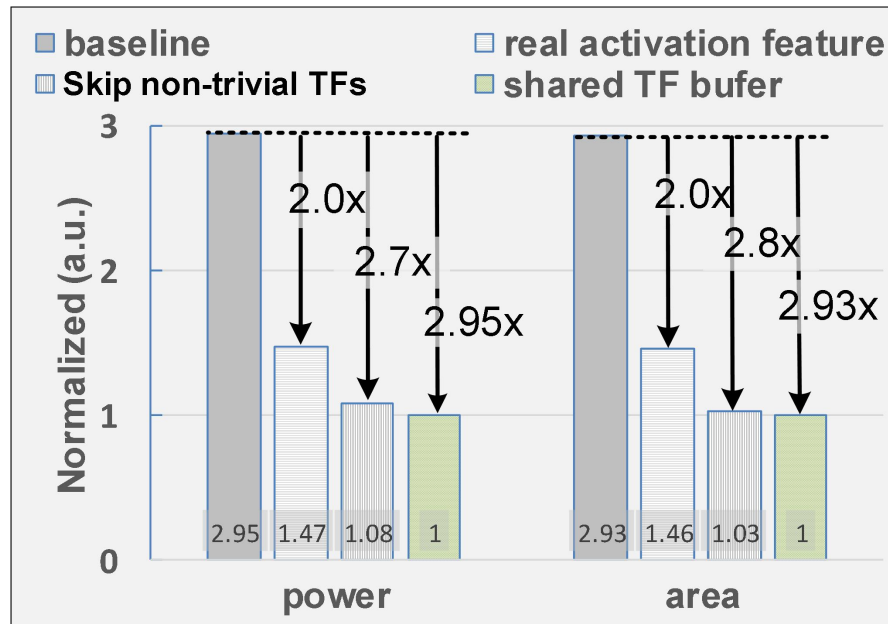


FFT Optimizing Results



Throughput varies for 1-12b FFT.

Power/area/performance improvement



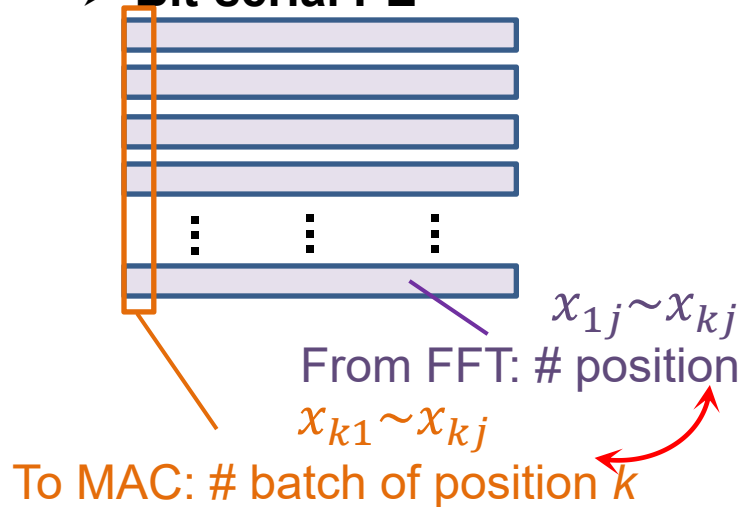
Power/area reduction

2-D Data-Reuse MAC Array

➤ 2-D data-reuse manner

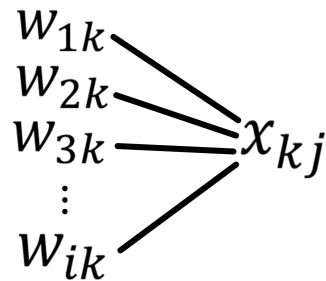
➤ TRAM transformation for data reuse

➤ Bit-serial PE

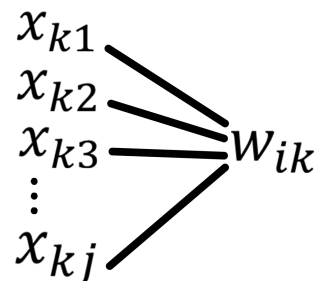


$$FFT(y_{ij}) = \sum_{k=1}^n FFT(w_{ik}) \odot FFT(x_{kj})$$

i, k : block row/col, j : activation batch.



Activation reuse



Weight reuse

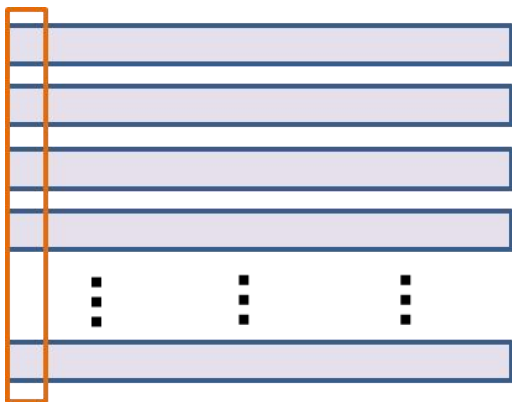
Require transposition

2-D Data-Reuse MAC Array

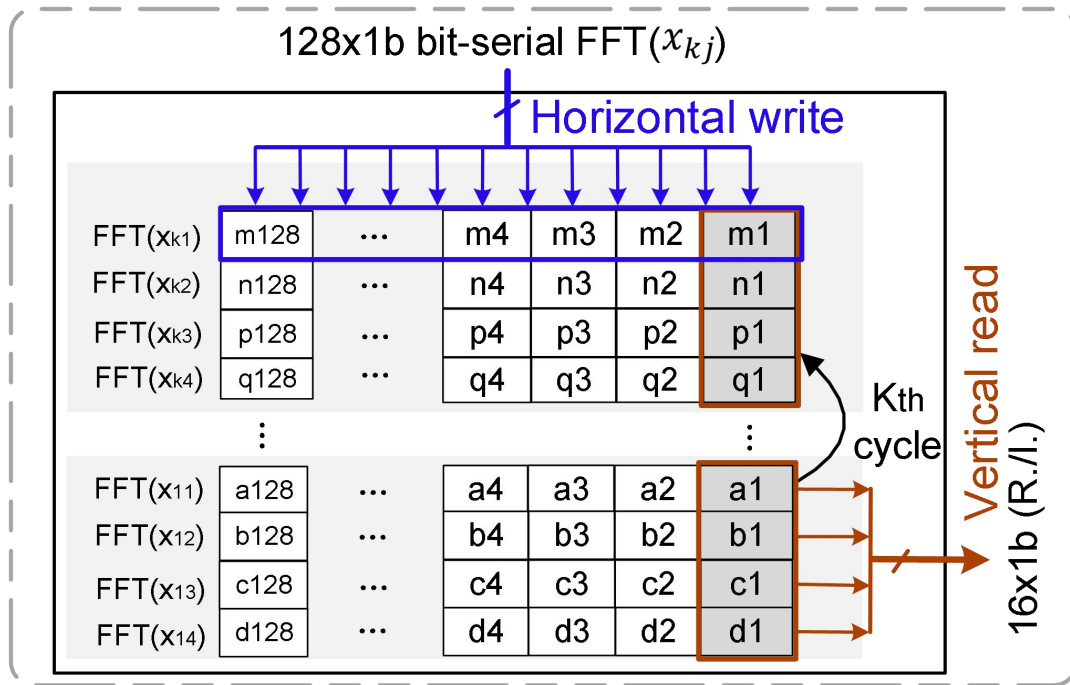
➤ 2-D data-reuse manner

➤ TRAM transformation for data reuse

➤ Bit-serial PE

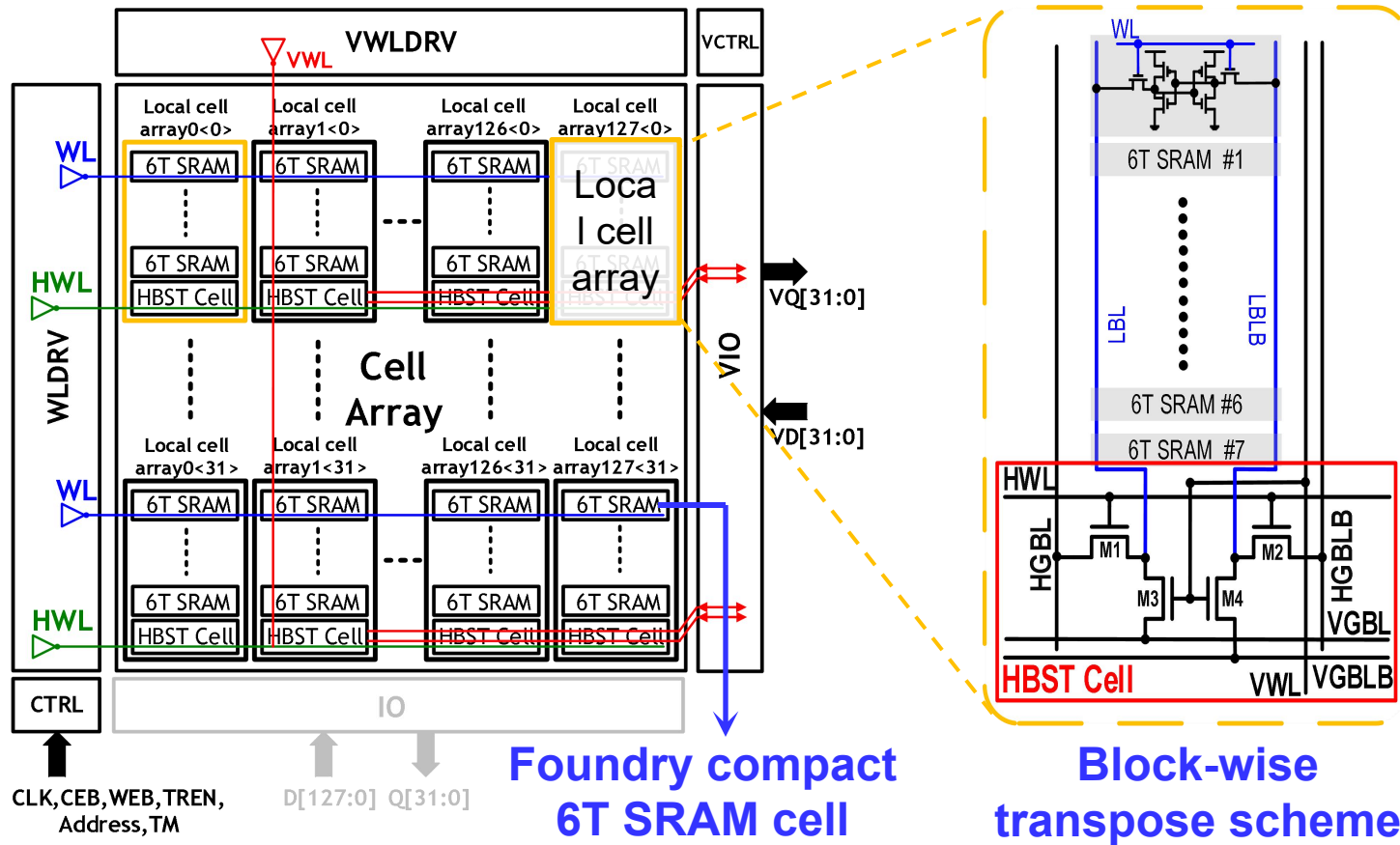


Data reuse conflict



Transpose SRAM (TRAM)

Hierarchical-Bitline-Switching (HBST) TRAM



HBST-TRAM Comparison

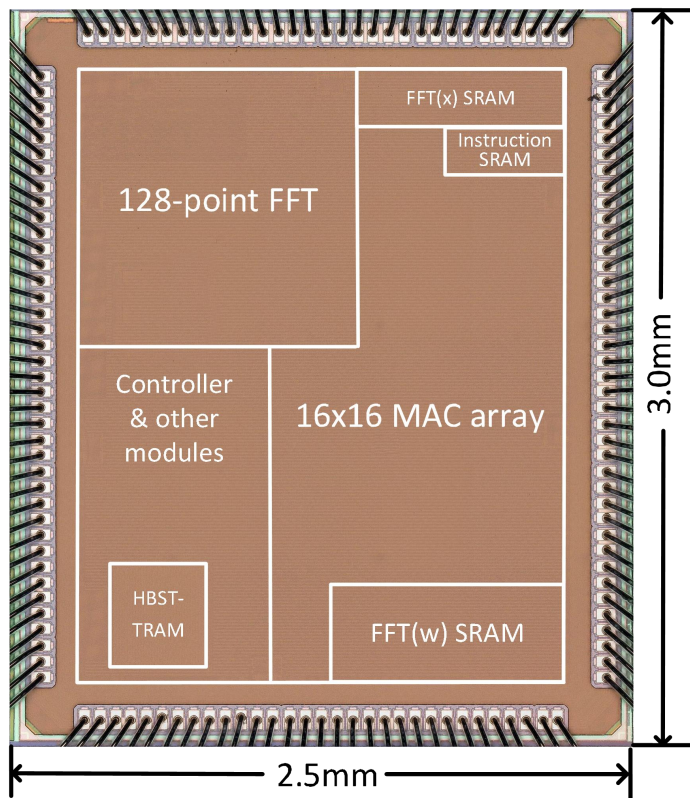
	This work	ISSCC 2017	CICC 2011
# of cell(block) transistors	6T (52T)	7T (56T)	8T (64T)
Transpose scheme	Block-wise	Cell-wise	Cell-wise
Transpose function	Read/Write	Read	Read/Write
Array size ^{*1}	1.45x (compact rule)	2.98x (logic rule)	5.9x (logic rule)
Delay ^{*2}	1x	2.09x	1.11x
Energy ^{*3}	1x	2.14x	1.29x

^{*1} Conventional compact 6T=1x

^{*2} at VDD=nominal, TT corner, 25° C, 32Kb macro, avg. of normal and transpose mode

^{*3} at VDD=nominal, FF corner, 85° C, 32Kb macro, avg. of normal and transpose mode

Chip Micrograph & Summary Table

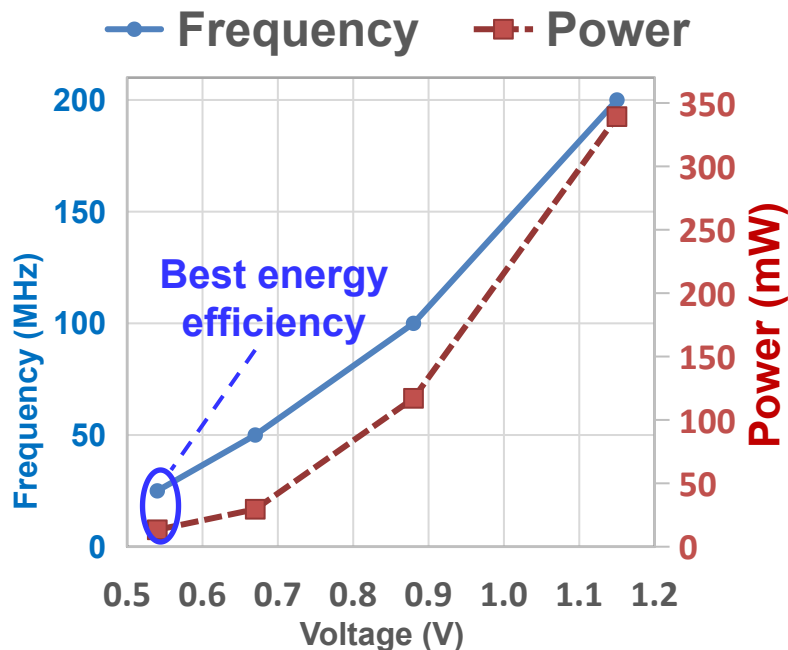


Technology	TSMC 65nm GP
Chip Area	3.0mmx2.5mm
Core Area	2.46mmx1.96mm
Support NN type	CNN+FC+RNN
Weight Bit-precision	1-12 bit
Activation Bit-precision	1-12 bit
Total SRAM	100kB
HBST-TRAM	4kB
Supply Voltage	0.54-1.15V
Frequency	25-200MHz
Power	13.3-339.2mW
Performance	0.13-14.9TOPS
Energy Efficiency ¹⁾	0.39-140.3TOPS/W

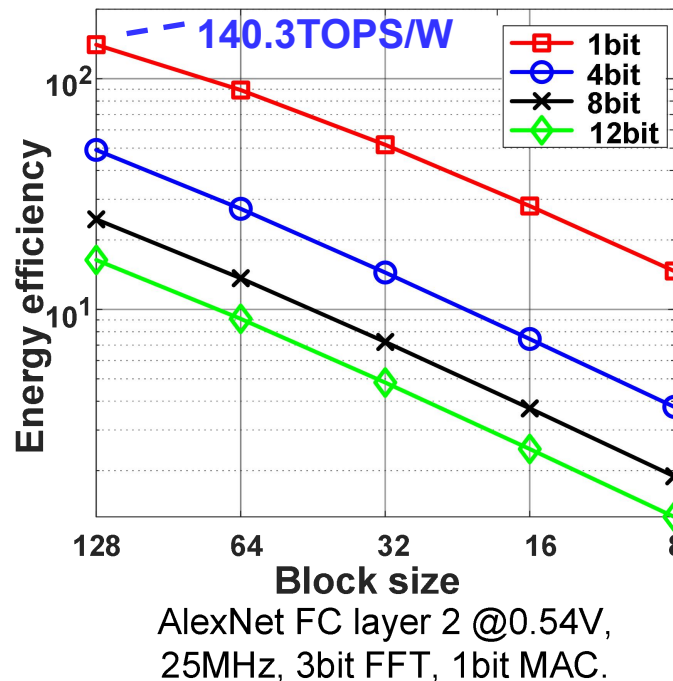
1) minimum @1.15V,200MHz,12bit, block size=8.
Maximum @0.54V,25MHz ,3bit FFT, 1bit MAC,
block size=128, on AlexNet FC layer 2.

Performance Scaling

Voltage-frequency scaling



Block-size/bit-precision scaling



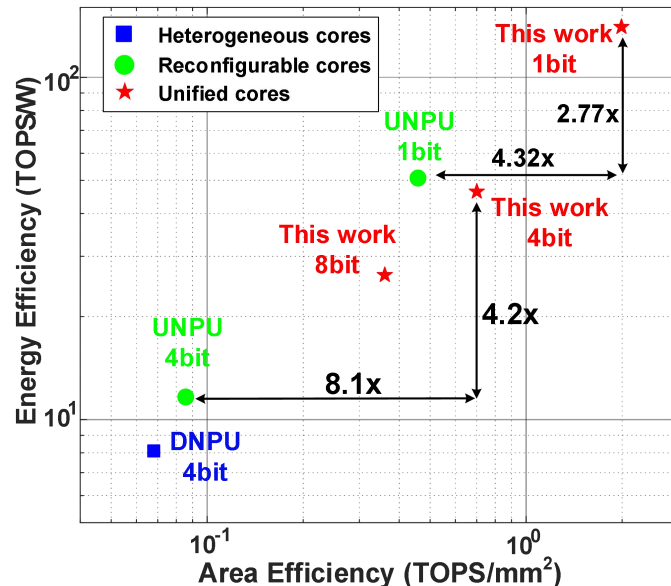
Performance Comparison

➤ Comparison with state-of-the-art NN processors

	ISSCC17-DNPU[2]	ISSCC18-UNPU[3]	VLSI18-STICKER[1]	This work
Technology	65nm	65nm LP	65nm GP	65nm GP
Area[mm ²]	16	16	12	7.5
Support NN type	CNN+FC+RNN	CNN+FC+RNN	CNN+FC	CNN+FC+RNN
Weight precision	4-7b(Quantized)	1-16b	8b	1-12b
Activation precision	4,8,16b	16b	8b	1-12b
Performance (TOPS)	1.08	7.37	5.6	14.9 ¹⁾
Power(mW)	34.6-279	3.2-297	20.5-248.4	13.3-339
Area efficiency (TOPS/mm ²)	0.068(4b) 0.79x	0.086(4b) 1x	0.47(Conv)	0.70(4b) 8.1x¹⁾
	-	0.46(1b) 1x	-	1.99(1b) 4.32x¹⁾
Energy efficiency (TOPS/W)	8.1(4b) 0.70x	11.6(4b) 1x	62.1 (Conv)	49.1(4b) 4.2x²⁾
	-	50.6(1b) 1x	-	140.3(1b) 2.77x²⁾

One operation (OP) represents one real multiplication or one real addition.

1) @1.15V,200MHz; 2) @0.54V,25MHz. AlexNet FC layer 2, block size 128.



Outline

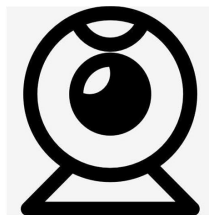
- ❑ Introduction to Neural Network Sparsity
- ❑ Spatial-Domain Sparsity Based NN Processor
- ❑ **Temporal-Domain Sparsity Based NN Processor**
- ❑ Sparsity-Aware CIM NN Processor
- ❑ Conclusion

Temporal Similarity in Video

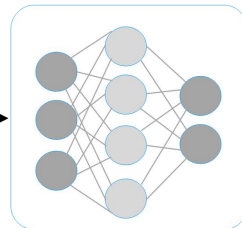
A Simple Video Case



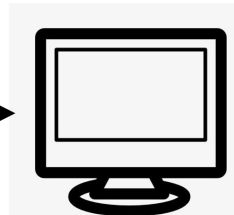
Camera



Neural Network



Display and Control



Frame 1

Similar



Frame 2

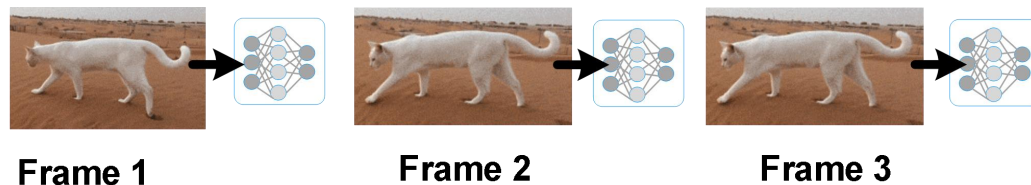
TimeLine

Sequential Frames in Video
are Similar to Each Other

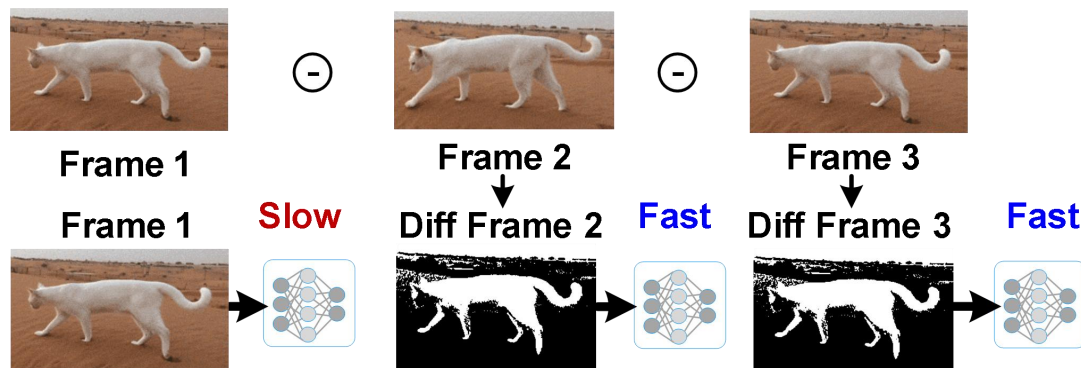
Question: IS There Any
Acceleration Space for
CNN?

Differential Frame CNN

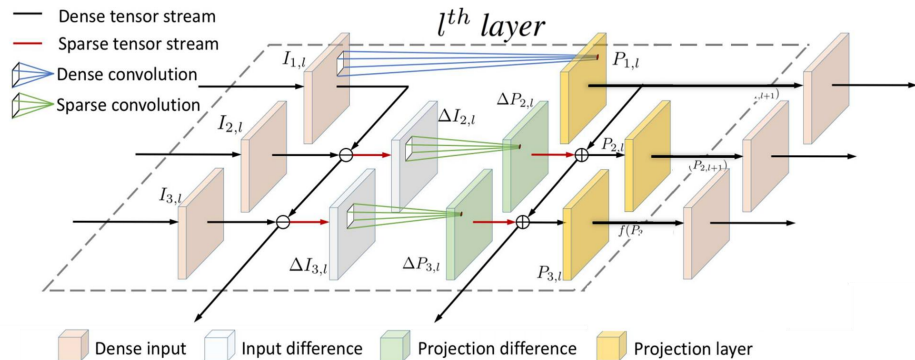
Traditional Way
Process Full Frame



New Dimension
Process Differential Frame



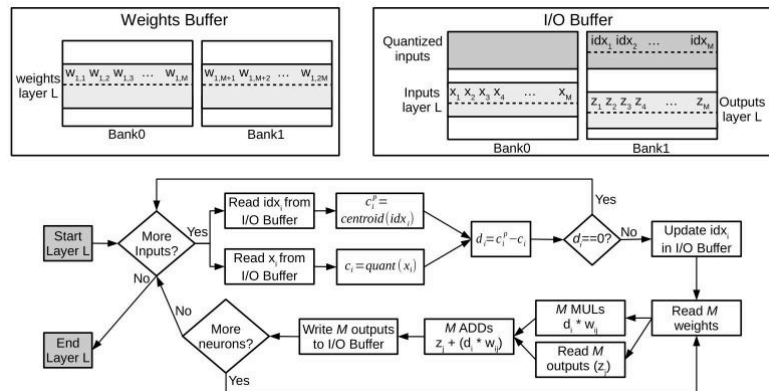
Related Works of Temporal Sparsity



Differential-Frame CNN Algorithm

BW Zhang @ CVPR2018

16%~37% Sparsity Improvement

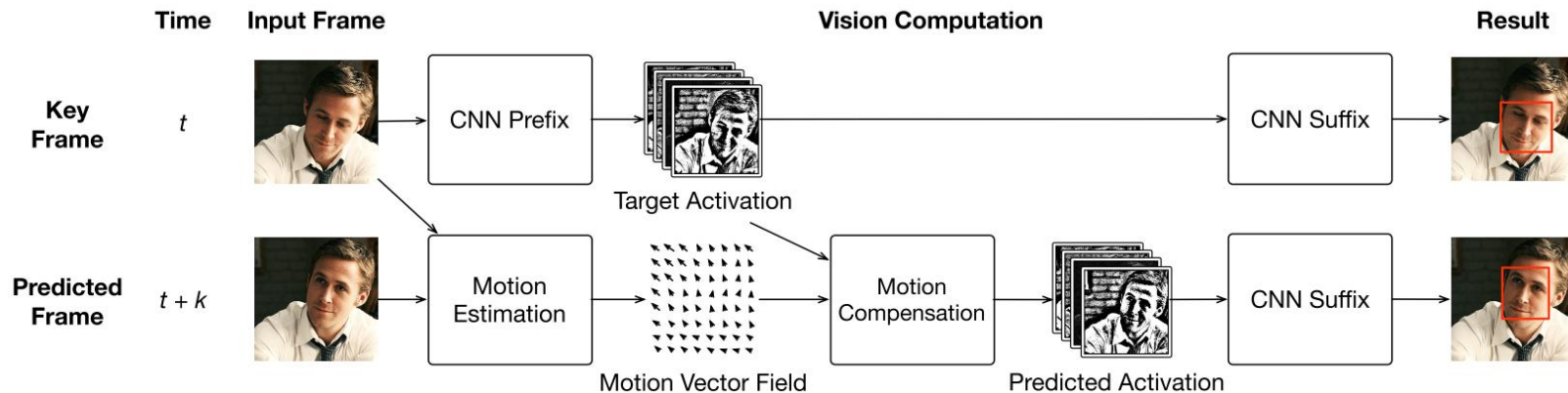


Differential-Frame CNN Accelerator

Marc Riera @ ISCA2018

Inter-Frame Data Reuse Architecture

Related Works of Temporal Sparsity



Activation Motion Compensation (AMC) for Video CNN

CNN $\leftrightarrow$ AMC (Motion Vector) + Prediction Error

54%-87% Energy Reduction and Up to 5% Accuracy Loss

Mark Buckler @ ISCA2018
YC Wang @ DAC2020

Observations of Temporal Sparsity

- **Viewpoint 1 : Sequential Frames are Similar but Not Identical**
- **Viewpoint 2 : Diff-Frame Activations Are **Not Sparse****
 - Diff-Frame Sparsity is no More than **60%** in Typical CNN Models
 - Only **11.7%~26.6%** Sparsity Improvement from the Diff-frame CNN
 - Higher Performance → **Accuracy Loss**

Network	Original Sparsity (Activation)	Diff-frame Sparsity (Activation)
AlexNet	39.7%	56.4%
VGG-16	30.3%	56.7%
MobileNet-V1	31.6%	43.3%

Sparsity: The ratio of zero data. Evaluate under a driving dataset.

Acceleration by Truncation:

- Higher Diff-frame Sparsity
- **Truncation Error**

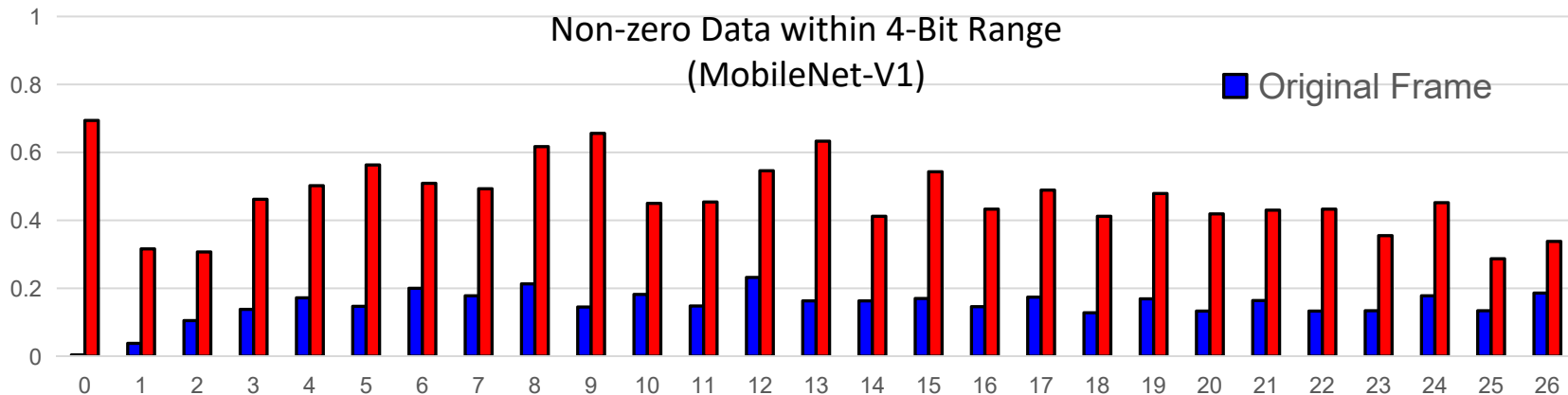
Acceleration by AMC:

- Skip CNN Computation
- **Prediction Error**

**These error will be
accumulated over frames !**

Observations of Temporal Sparsity

- **Viewpoint 1 : Sequential Frames are Similar but Not Identical**
- **Viewpoint 2 : Diff Frames Are **Not Sparse****
- **Viewpoint 3 : Diff Frames Have More **Narrow Distribution****
 - High Bit Sparse : Only **<10%** Data Exceed the 4-Bit Range
 - Most of Diff-frame Activations Distribute in Lower Bit Range



Observations of Temporal Sparsity

- **Viewpoint 1 : Sequential Frames are Similar but Not Identical**
- **Viewpoint 2 : Diff Frames Are **Not Sparse****
- **Viewpoint 3 : Diff Frames Have More **Narrow Distribution****

Challenge of Temporal Sparsity:

How to Efficiently Process the Narrow Distributed Video Frames without Loss of Accuracy?

Hybrid-Precision Diff-Frame CNN

- **Use Inter Frame Similarity**

- Get Diff Frame First
- Diff Frames are not Sparse

- **Hybrid-Precision**

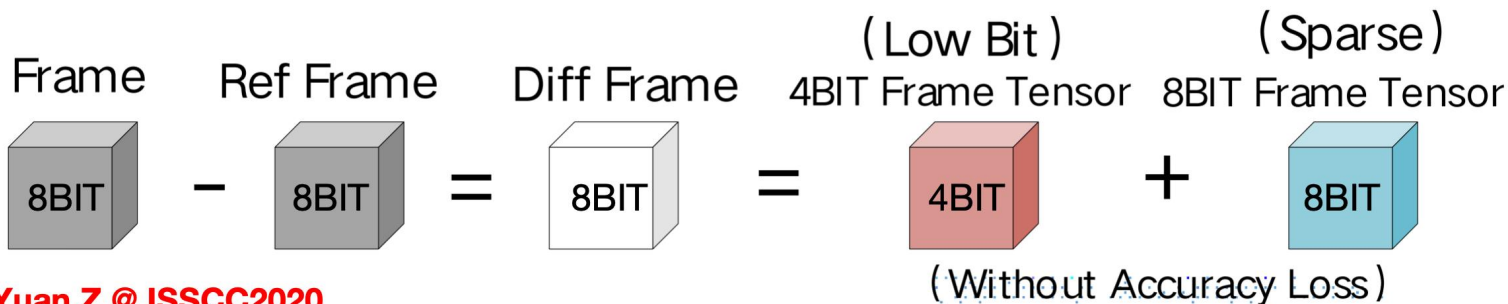
- A **Low-Bit** Tensor
- A **High-Bit Sparse** Tensor
- No Accuracy Loss



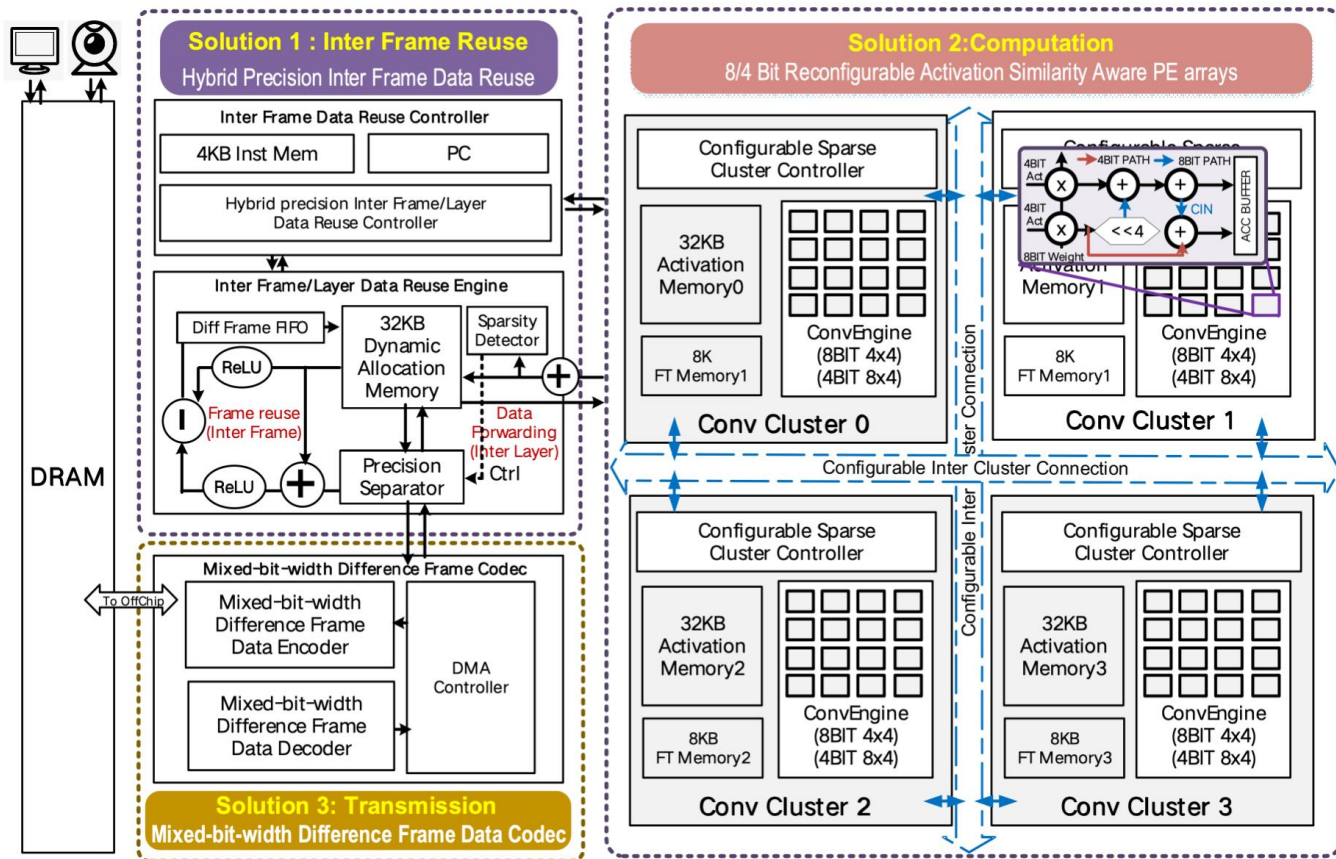
Original Frame



Diff Frame

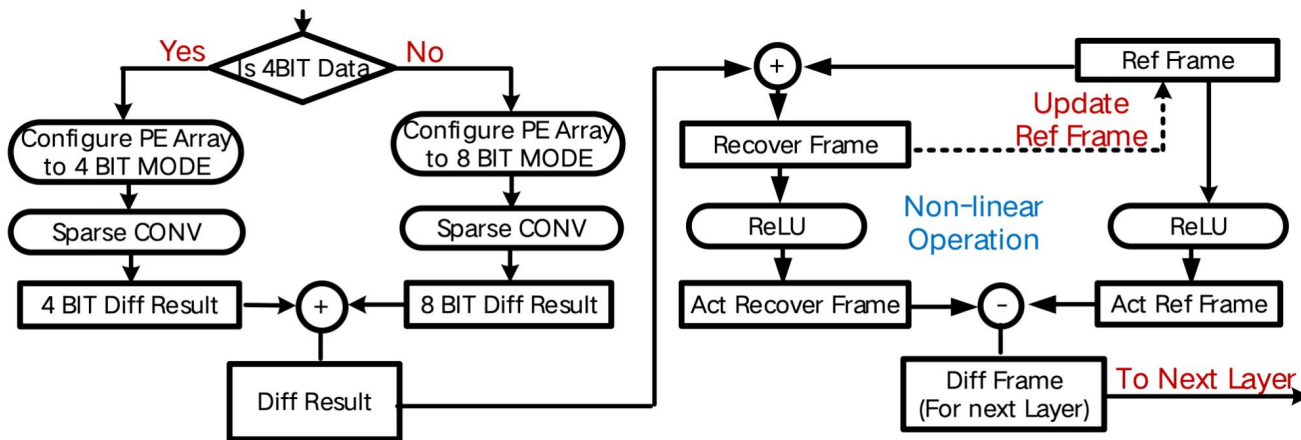
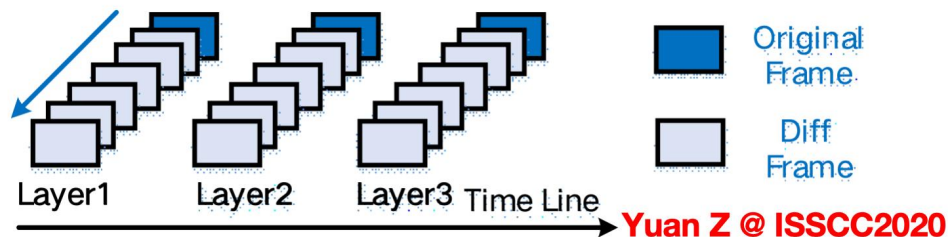


Activation Similarity Aware CNN Video Processor



Inter-frame Data Reuse Architecture

- Network Process Flow
- Diff Layer Process Flow
 - 2 Mod Sparse Conv Array
 - Get Diff Frame and Ref Frame



Inter Frame Data Reuse

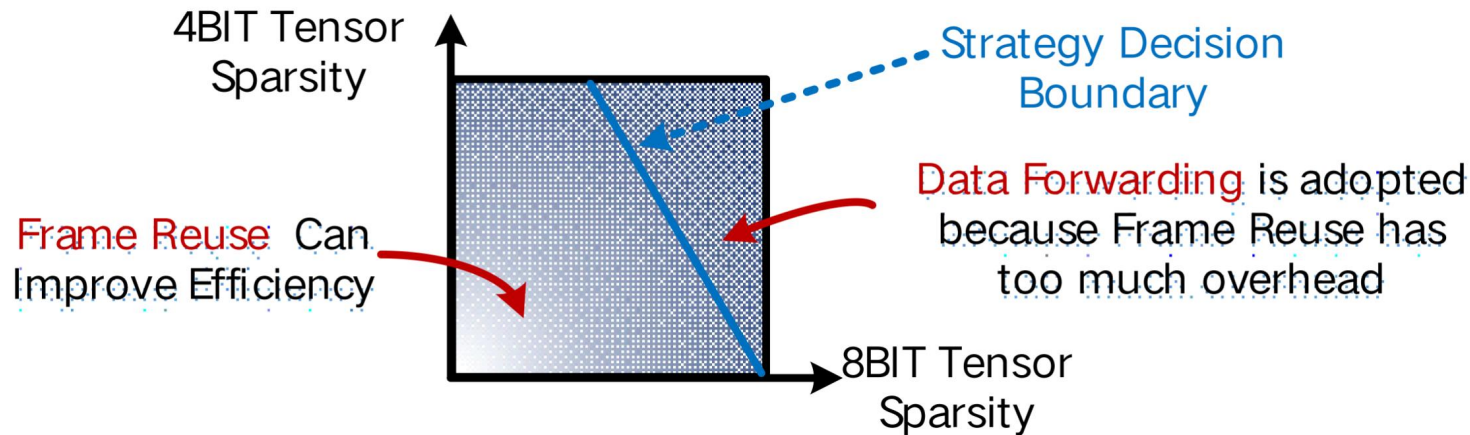
- **Theoretical Improvement Analysis**

$$\text{Performance Improvement} = 1 / [\text{Sparse (8BIT)} + \frac{1}{2} \text{Sparse(4BIT)}]$$

- **Process Strategy Trade-Off**

- **Method 1: Use Inter Frame Similarity with Diff Frame**

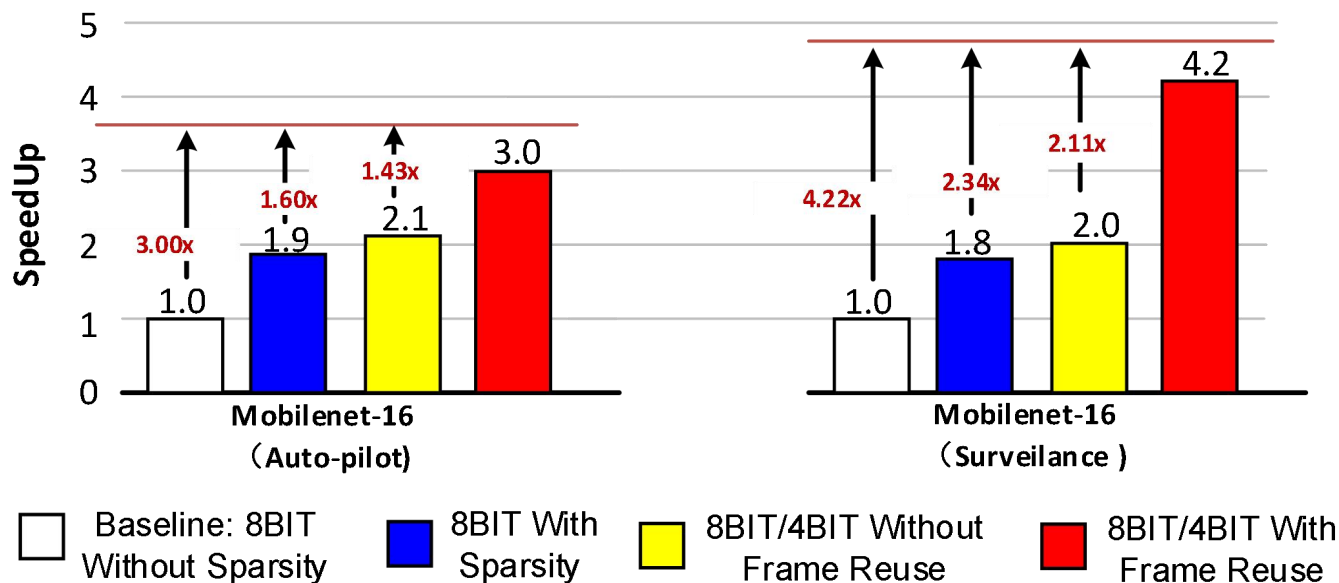
- **Method 2: Directly Process Original Frame**



Theoretical Performance

➤ Theoretical Improvement Analysis

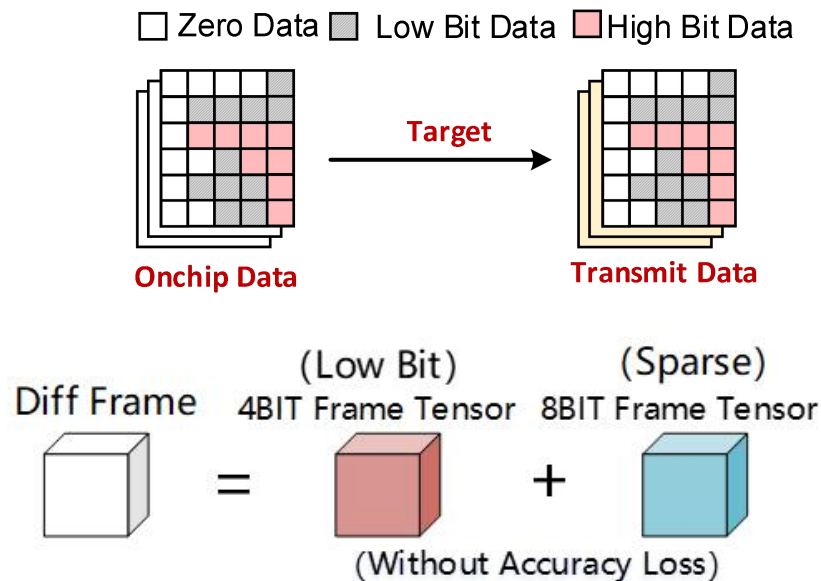
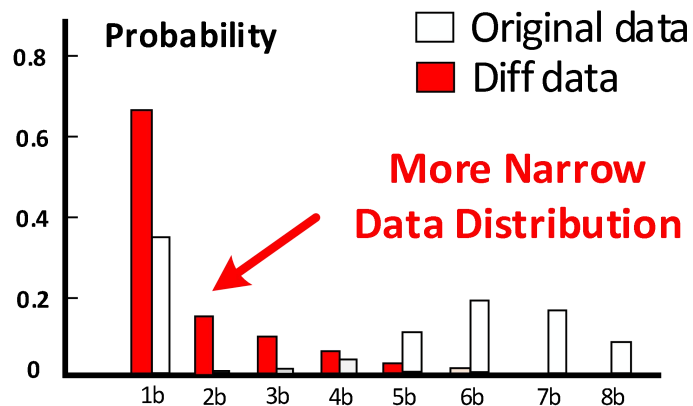
➤ Performance Improvement = $\frac{1}{(1 - \text{Sparsity}_{8B}) + \frac{1}{2} * (1 - \text{Sparsity}_{4B})}$



Mix-bit-width Difference Frame Data Codec

➤ Motivation

- Use Diff Frame Feature : **Narrow Data Distribution**
- Support Conv Engine : **High Bit Tensor / Low Bit Tensor**



Temporal Sparsity for Activations I/O

➤ Mix-Strategy Diff-frame Coding

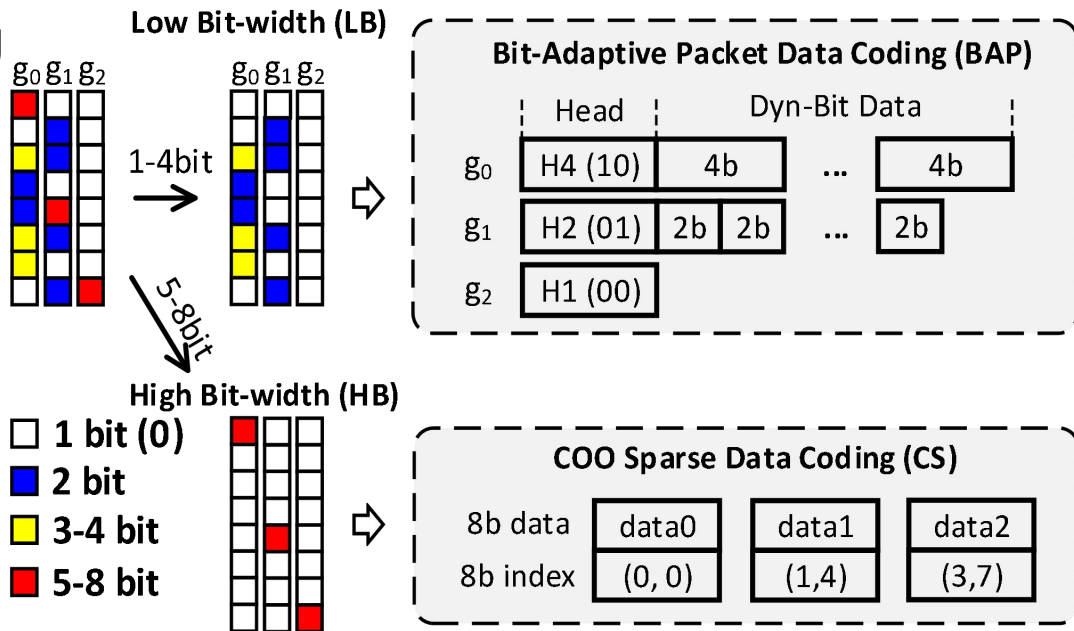
➤ Diff-frame Data Transmission

➤ Low Bit Data Coding

- Quantize Data in Packet
- 1b/2b/4b Dynamic Bit-width

➤ High Bit Data Coding

- COO Sparse Coding
- Data + Sparse Index



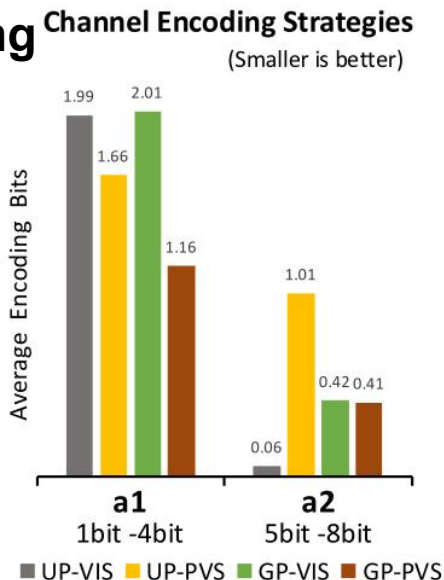
Temporal Sparsity for Activations I/O

➤ Mix-Strategy Diff-frame Coding

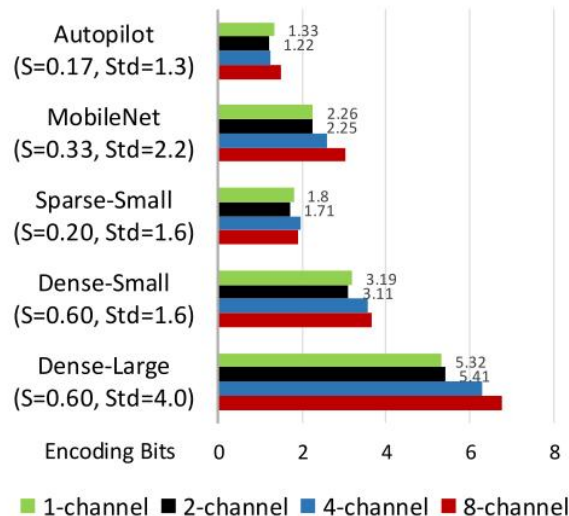
- Diff-frame Data Transmission
- Low Bit Data Coding
- High Bit Data Coding

➤ Coding Design Optimization

- Multiple Channels Encoding
- Sparsity-Quantization Adaption
- Channel Coding Selection



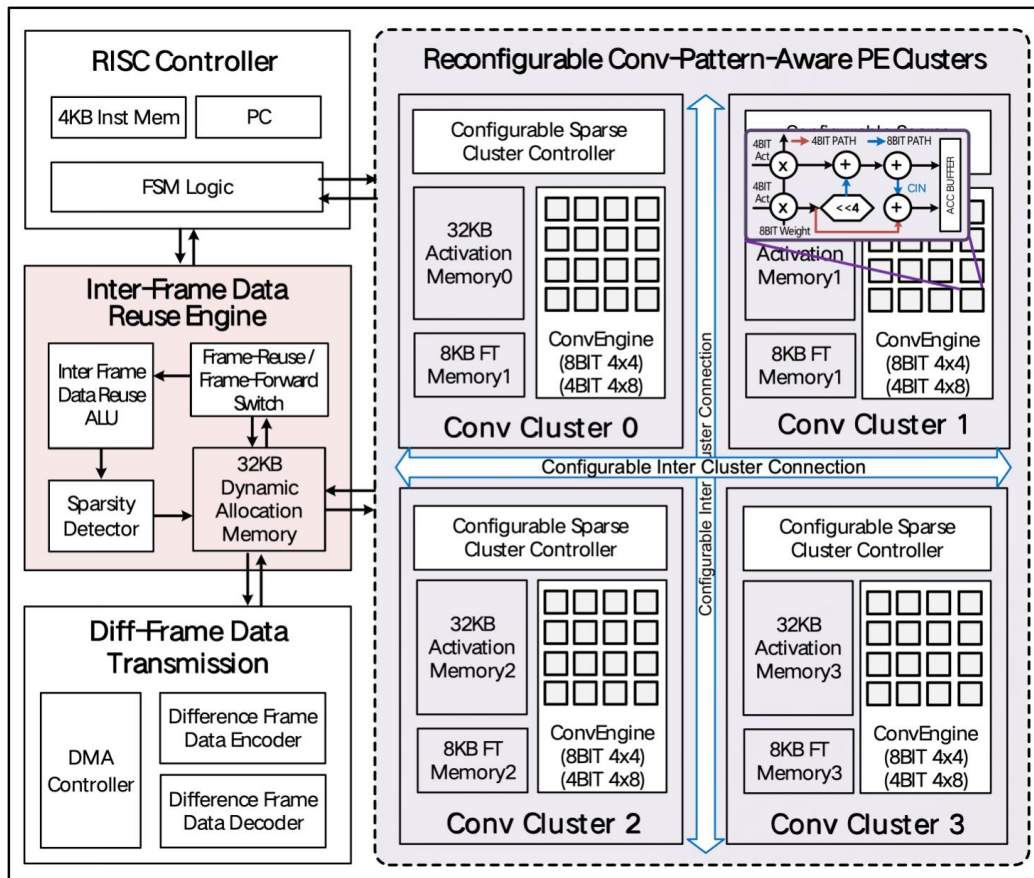
Multi-Channel Schemes Comparison



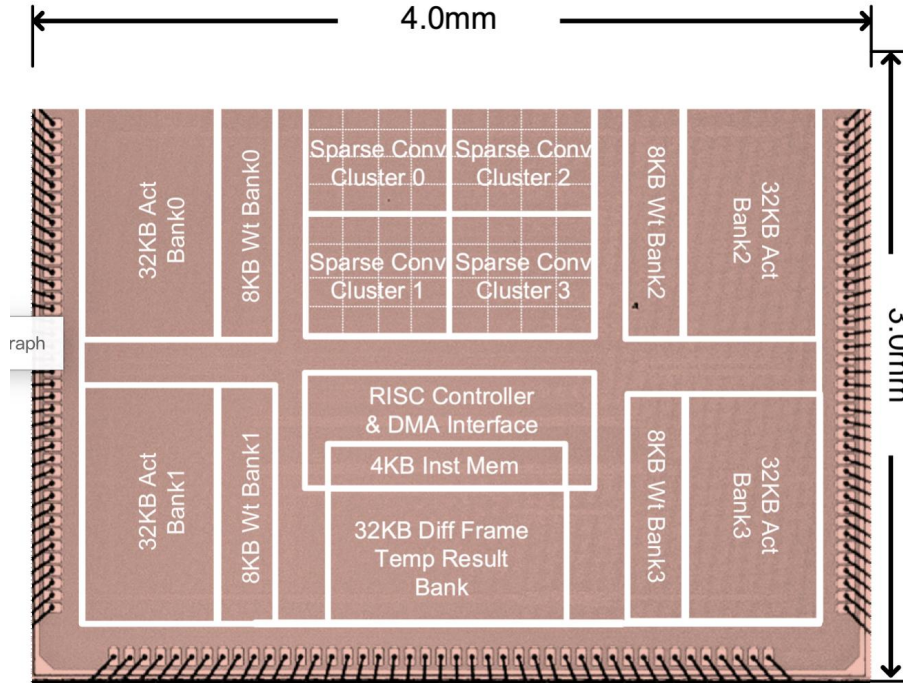
2.2x-18.2x Activation Compression in Three Scenarios

Yang YX @ ISLPED2020

Video AI Chip Implementation



STICKER-V Chip Micrograph



Chip Photograph

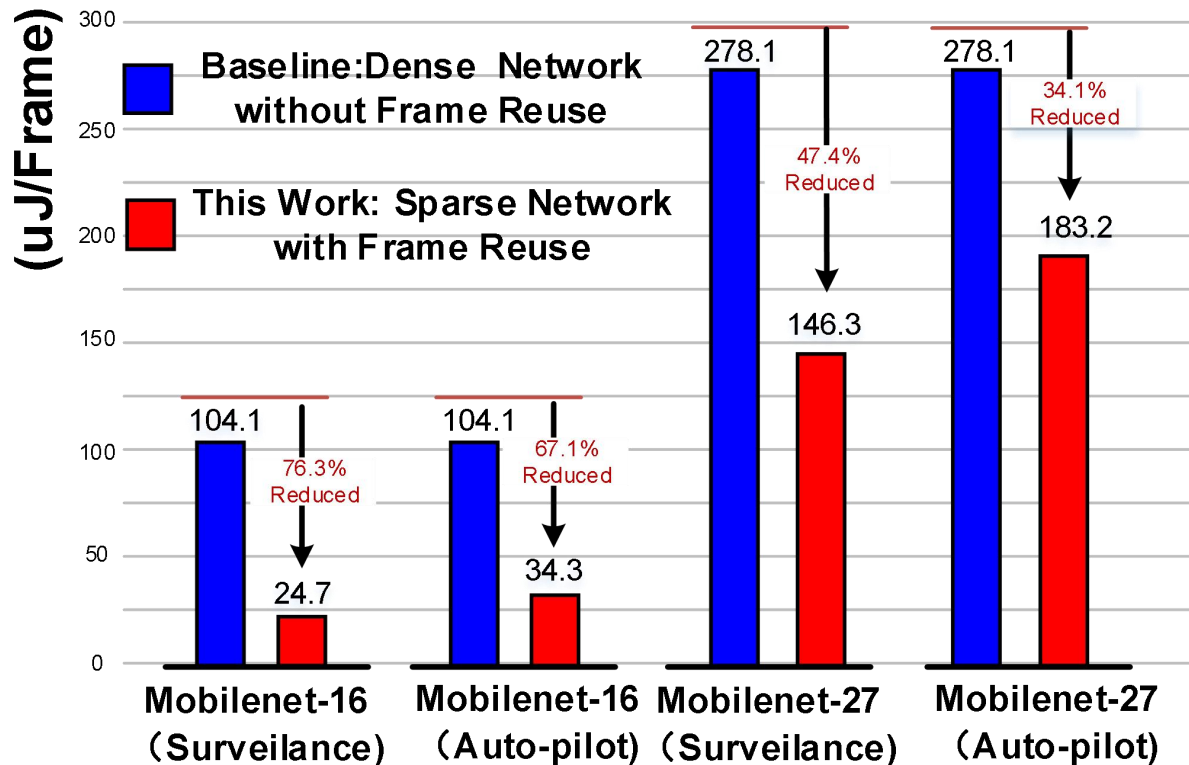
Technology	TSMC 65nm GP
Chip Area	4.0mmx3.0mm
Activation Length	8bit, 4bit
Weight Length	8bit
MAC	64(8BIT)/128(4BIT)
Total SRAM	196KB
Supply Voltage	0.52-0.99V
Core Frequency	25MHz-100MHz
Power Consumption	7.3-99.0mW
Energy Efficiency	0.129-13.3TOPS/W
Frame Energy	24.7-183.2 μ J/Frame

* Efficiency Varies with Different
Voltage/Frequency/Inter Frame Data Reuse

Chip Summary

Energy Efficiency of NN Processor

Video AI Core Frame Energy



Comparison Table

Work	Year	Tech	Core Area	Supply Voltage	Sparse Conv Support			Data Reuse Support		Power	Efficiency	Frame Energy
		(nm)	(mm ²)	(V)	Normal	DW	PW	Forwarding	Inter Frame	(mW)	(TOPS/W)	(μ J/Frame) ¹
EYERISS[1]	ISSCC2016	65	12.3	0.82-1.2	x	x	x	x	x	235-332	0.24-0.31	-
ENVISION[2]	ISSCC2017	28	1.9	0.65-1.1	x	x	x	o	x	8-300	0.53-10	-
STICKER[3]	VLSI2018	65	7.8	0.67-1.0	o	x	x	o	x	21-248	0.41-62.1	-
UNPU[4]	ISSCC2018	65	11.6	0.63-1.1	x	x	x	o	x	3-297	3.08-50.6	-
STICKER-T[5]	ISSCC2019	65	7.5	0.54-1.1	x	x	x	o	x	13.3-339.2	0.39-140.3	-
Samsung[6]	ISSCC2019	8	5.5	0.50-0.8	o	x	x	o	x	39-1553	4.5-11.5	-
This Work	-	65	7.8	0.51-1.0	o	Empty Channel	o	o	o	7.3-99	0.129-13.3 ²	24.7-183.2

1. Mobilenet-16 and Mobilenet-27 are trained for Auto-pilot and surveillance cameras applications.

2. Minimum efficiency is under 0.99V/100M/without any data reuse. Maximum efficiency is under 0.58V/50M/15% Nonzero Weight/ 75% Inter frame Data Reuse.

Summary of Temporal Sparsity NN Processor

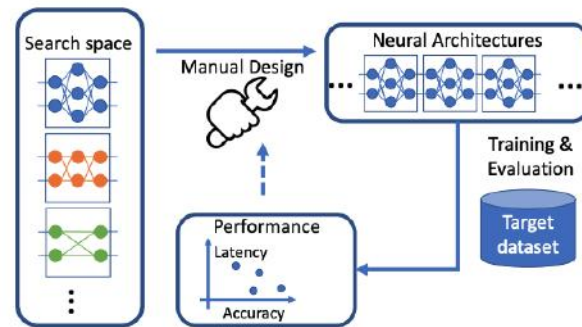
- **A New Dimension: Accelerate CNN by Temporal Sparsity**
 - CNN activation will be similar between frames
 - Diff-frame CNN can reuse previous result to save computation
- **Diff Frames Are Not Sparse, but Have Narrow Distribution**
 - Diff-frame sparsity is above 40%, only the high-bit data is very sparse
 - Most Diff frame data only requires no more than 4 bits precision
- **Acceleration of Hybrid-Precision inter-frame Data Reuse**
 - Separately process the low-bit and high-bit of the Diff frame
 - Reuse the previous-frame result in on-chip memory
 - I/O Compression by Diff-frame data coding

Outline

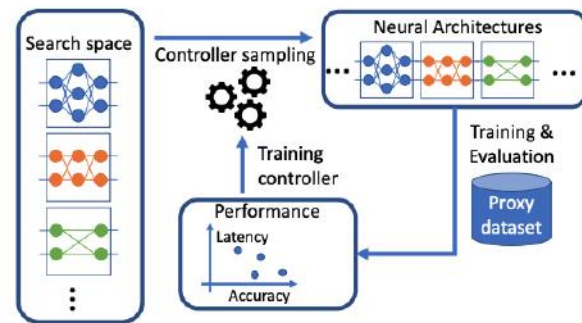
- ❑ Introduction to Neural Network Sparsity
- ❑ Spatial-Domain Sparsity Based NN Processor
- ❑ Temporal-Domain Sparsity Based NN Processor
- ❑ **Neural Architecture Search**
- ❑ Conclusion

Neural Architecture Search

- Manually determine the network architecture is a tedious process.
 - Large number of hyperparameters, e.g., # of layers, connections between layers, and types of layers
- NAS is proposed to automatically search for the optimal architecture at the cost of more computation.

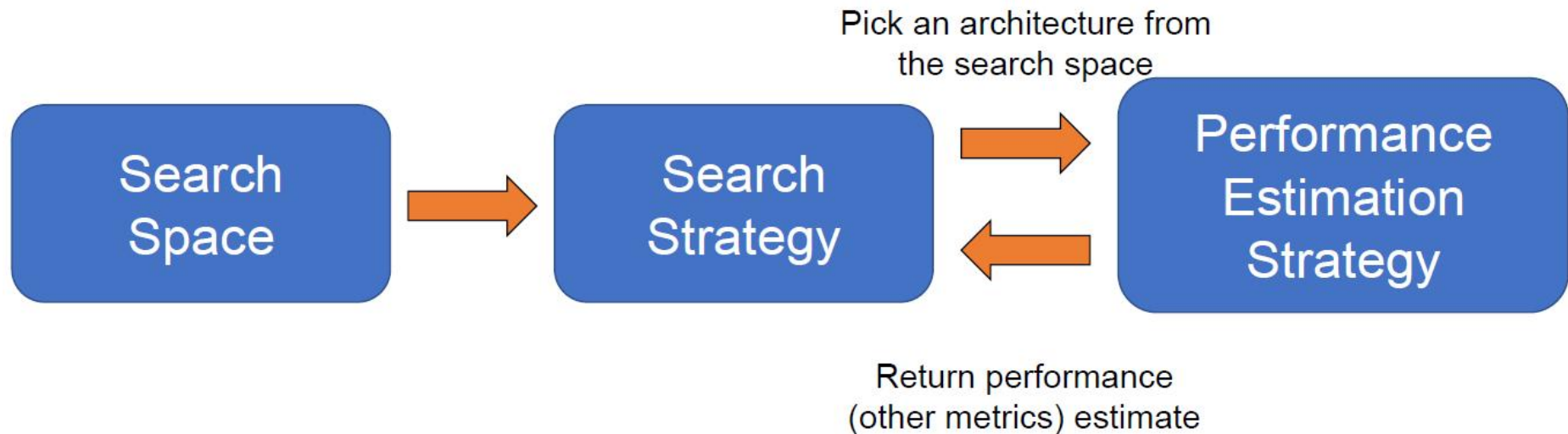


(a) A typical flow of manual ConvNet design.

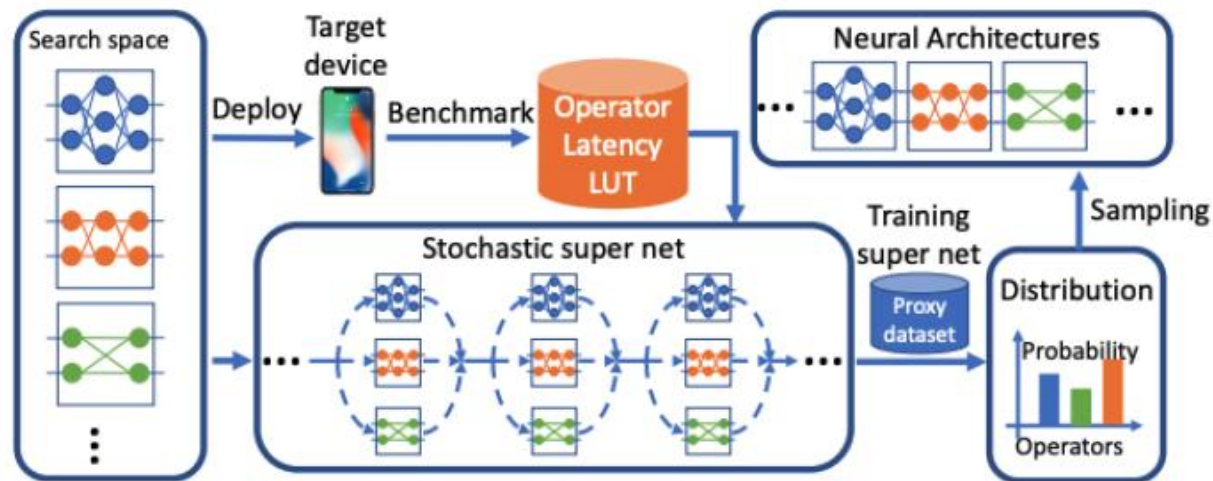


(b) A typical flow of reinforcement learning based neural architecture search.

Neural Architecture Search



Neural Architecture Search



Model	#Parameters	#FLOPs	Latency on iPhone X	Latency on Samsung S8	Top-1 acc (%)
FBNet-iPhoneX	4.47M	322M	19.84 ms (target)	23.33 ms	73.20
FBNet-S8	4.43M	293M	27.53 ms	22.12 ms (target)	73.27

Table 5. FBNets searched for different devices.

Conclusion

- ☐ **Sparsity overview**
- ☐ **By designing a domain-specific accelerator, we can achieve better performance.**
- ☐ **SW/HW codesign philosophy**
- ☐ **NAS is another choice to design efficient network**